

AI LEARNING NOTES

Teaching a Language Model to Solve Sudoku

An End-to-End Fine-Tuning Journey on Apple Silicon

Using LoRA (a technique that trains a tiny fraction of a model's weights) and `mlx_lm`

Written for a reader who has never trained a machine learning model.

Every technical term is explained on first use. Glossary at the end.

Authored By: Douglas Mun with AI Assistance

Date: May 17, 2026

Classification: TLP: CLEAR - Public, Unrestricted Distribution

Hardware: MacBook Pro with M5 Max chip, 128 GB unified memory

Base Models: Qwen/Qwen3-1.7B → Qwen/Qwen3-4B

Training Framework: `mlx_lm.lora` (Apple Silicon native)

Inference Server: `vllm-mlx` (port 7001)

Final Result: 43 % overall accuracy — Easy 81 %, Medium 33 %, Hard 0 %

Executive Summary

This document chronicles my end-to-end attempt to post-train an open-weights language model using only a consumer Apple Silicon laptop and freely available tools. The goal was to teach a model to solve Sudoku puzzles — a problem with clear, verifiable rules — and play those solved puzzles in an interactive game.

Starting from Qwen/Qwen3-1.7B (1.7 billion parameters), seven training runs were conducted over two days. Each run revealed a new lesson about what goes wrong when teaching a model a structured task. By Run 6, the 1.7B model reached 45 % accuracy on easy puzzles. Scaling to 4B parameters (Run 7) pushed Easy accuracy to 81 %, though Medium (33 %) and Hard (0 %) puzzles remained a challenge.

Key Findings

- Dataset format is everything. A single wrong key name ('text' instead of 'prompt'/'completion') caused an entire run to learn nothing.
- Learning rate decay was essential in this experiment. A fixed learning rate (the step size the model uses when adjusting its weights) caused the model to 'unlearn' good weights after its best point. A cosine decay schedule — which automatically shrinks the step size over time so the model makes increasingly fine adjustments — prevented this. Other fine-tuning projects may succeed with different schedules; what matters here is that a decaying schedule clearly outperformed a fixed one.
- More data beats more iterations. Scaling from 5,000 to 20,000 training examples improved the best validation loss from 0.028 to 0.019 — a data ceiling, not a compute ceiling.
- Bigger models help with easy puzzles. The 4B model scored 81% Easy vs ~45% for 1.7B.
- Hard puzzles need a different inference strategy. Neither more data nor a larger model moved Hard accuracy above 0 %. Single-pass left-to-right generation cannot backtrack — changing the inference procedure, not just the training data, is likely required.
- Small- to mid-sized models can be fine-tuned locally with LoRA. This experiment demonstrates that on high-memory Apple Silicon hardware, the full pipeline — training, serving, and evaluation — runs on a single laptop in under 2 hours, with no cloud compute required for models of this size.

Before We Begin: Five Concepts in Plain English

This document assumes no prior machine learning knowledge. These five concepts appear on almost every page — understanding them now will make everything that follows much clearer.

Concept — Language Model: A language model is a program trained on billions of pages of text from the internet and books to predict the next word. When you type a sentence, it fills in the rest. Doing this trillions of times across vast text taught it to capture grammar, facts, and reasoning patterns. It knows what words typically follow other words in almost any context. Models like GPT-4, Claude, and the Qwen3 model used here are all language models.

Concept — Token: A token is the basic unit of text that a language model reads and writes. It is not always a full word — it can be a word fragment, a punctuation mark, or a space. The word 'Sudoku' is one token; 'impossible' might be split into 'im' + 'possible' (two tokens). Models have a maximum number of tokens they can process at once — called a context window. In this project, `max_seq_length: 1024` means each training example can be at most 1,024 tokens long. A Sudoku puzzle + solution uses only ~200 tokens, well within that limit.

Concept — Training: Training is the process of adjusting a model's numerical weights to reduce its prediction errors on a given dataset. Each training step: (1) feed examples to the model; (2) measure how wrong the predictions are (the loss); (3) calculate which direction to adjust each weight to reduce the loss (this calculation is called a gradient); (4) take a small step in that direction (gradient descent). Repeat thousands of times. Pre-training — where base models like Qwen3 are built — uses this process on billions of examples and takes weeks on thousands of GPUs. Fine-tuning uses the same process on a small task-specific dataset and takes minutes to hours.

Concept — Inference: Inference is what happens when you actually use a trained model to generate text, as opposed to training it. When you send a Sudoku puzzle to the model, and it responds with a solution, that is inference. The model's weights are frozen during inference; nothing is learned. In this project, inference is handled by a small HTTP server (`vllm-mlx`) that accepts puzzle requests and returns solutions.

Concept — HuggingFace: HuggingFace is a platform (huggingface.co) where researchers and companies publish open-source AI models for free download. Think of it as GitHub but for trained model weights. The base models used here — Qwen/Qwen3-1.7B and Qwen/Qwen3-4B — are downloaded automatically from HuggingFace by the training framework the first time you run training. The files are cached in `~/.cache/huggingface/hub/`, so you only download once.

One loop runs throughout this entire document. Keep this in mind as you read:

Term	What does it mean in this project
Prompt	What the model sees as input is the Sudoku puzzle.
Completion	What the model must learn to produce is the solved 9×9 grid.
Loss	How wrong the completion is. A number to minimise during training.
Validation	Loss measured on examples the model has never seen — the honest test.
Accuracy	The separate task check: does the full 9×9 grid satisfy all Sudoku rules?

1. Purpose of This Document

This document serves three purposes:

1. Educational record. A step-by-step account of adapting a pre-trained language model for a new task (called fine-tuning) from zero prior experience. Every mistake is documented alongside the fix.
2. Technical reference. Exact commands, configuration files, and code snippets are included to enable the experiment to be reproduced.
3. Lesson log. Each training run produced a lesson. These lessons are collected in plain language to help readers avoid the same pitfalls.

Sudoku was chosen because it has perfectly verifiable answers — there is no ambiguity about whether a solution is correct. This makes it an ideal benchmark for measuring whether a model has truly learned a task versus merely memorised patterns.

A traditional backtracking Sudoku solver is simpler, faster, cheaper, and perfectly accurate. That is not the point. This project uses Sudoku as a controlled laboratory: a task where outputs are structured, verifiable, and unforgiving — so that every training decision has a measurable consequence. In production, the correct engineering answer is the symbolic solver; in education, the value is that Sudoku makes fine-tuning failure modes visible. The educational goal is to understand how fine-tuning behaves, not to build the best Sudoku solver.

2. Environment Setup

2.1 Hardware

All training, serving, and evaluation ran on a single machine: Apple MacBook Pro with an M5 Max chip and 128 GB of unified memory, running macOS 26 (Darwin 25.4.0).

Concept — Unified Memory: Apple Silicon integrates the CPU, GPU, and Neural Engine (ANE) on a single chip, sharing a single pool of RAM. Unlike discrete GPUs (e.g., NVIDIA RTX 4090 with 24 GB VRAM), there is no separate video memory limit. With 128 GB unified memory, models up to ~70 GB fit comfortably. Training the 1.7B model used only 18.5 GB of peak memory.

2.2 Software Stack

Component	Version / Detail
macOS	26 (Darwin 25.4.0)
Python	3.12.x (arm64, Homebrew)
mlx	0.24.x
mlx-lm	0.22.x
vllm-mlx	latest (pip install vllm-mlx)
pygame	2.6.x
requests	2.32.x
python-docx	1.1.x

2.3 Installation Commands

```
# Create arm64 virtual environment
$(brew --prefix python@3.12)/bin/python3.12 -m venv ~/Develop/vllm-mlx
source ~/Develop/vllm-mlx/bin/activate

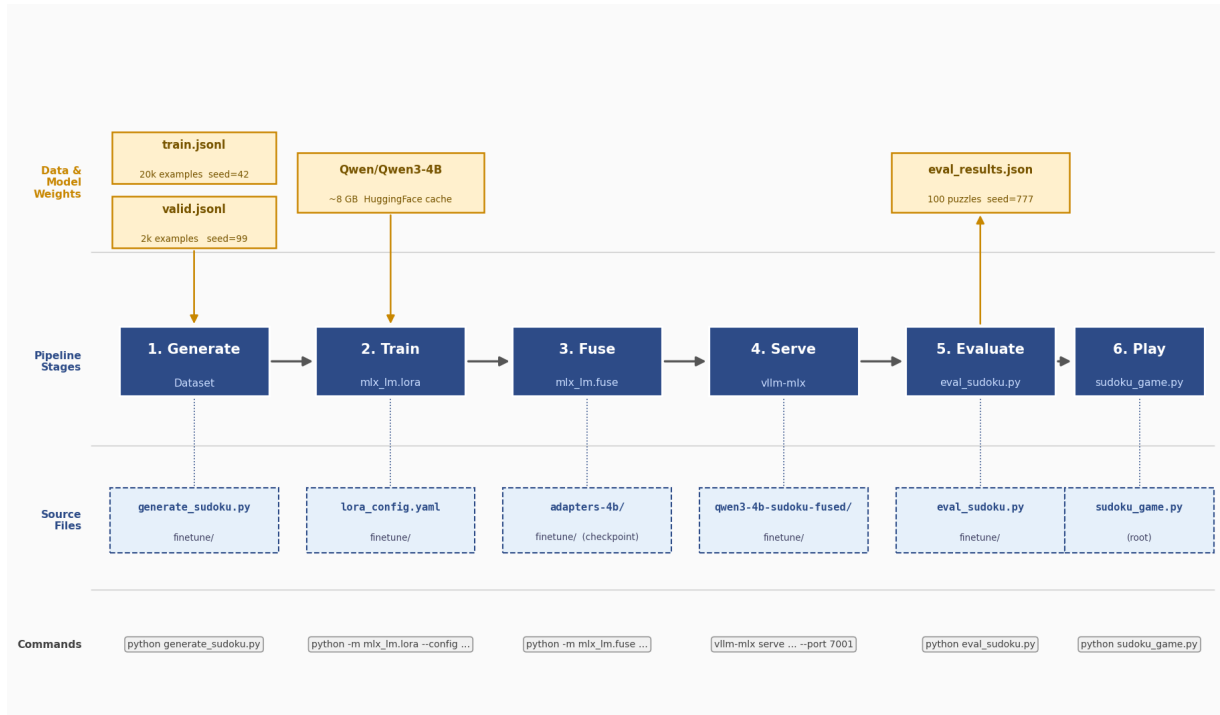
# Install dependencies
pip install vllm-mlx mlx-lm pygame requests
```

2.4 Pipeline Architecture & Key Files

The diagram below shows the full end-to-end pipeline, with the key file at each stage and the commands connecting them.

Figure 0 — Full training pipeline: from dataset generation through to the playable game.

2.5 Training Framework: mlx_lm.lora



`mlx_lm` is Apple's open-source library for running and adapting language models on Apple Silicon. It is built on top of MLX — Apple's own numerical computing framework designed specifically for the unified memory architecture of M-series chips. The '.lora' suffix refers to LoRA (Low-Rank Adaptation), the specific fine-tuning technique used — both terms are explained fully in Section 4.

Concept — MLX (Apple's Machine Learning Framework): MLX is Apple's open-source array framework for machine learning on Apple Silicon. It is optimised for unified-memory architecture and accelerates computation through Apple's native GPU/Metal stack. Standard deep learning frameworks like PyTorch run on NVIDIA GPUs via the CUDA driver; on Apple Silicon, there is no CUDA. MLX uses the M-series GPU via Metal to achieve native performance without a translation layer. (Note: not every MLX operation necessarily uses the ANE — workload routing depends on Apple's runtime and is not directly observable.)

`mlx_lm.lora` is the specific command within `mlx_lm` that handles LoRA fine-tuning (adapting a pre-trained model using small trainable matrices — explained fully in Section 4). It reads a YAML configuration file, downloads the base model from HuggingFace on first run, constructs the adapter layers, and runs the training loop — printing error metrics every N iterations and automatically saving progress snapshots (checkpoints).

Why not PyTorch / Hugging Face Trainer on Mac?

The standard model-adapting stack (Hugging Face Transformers + PyTorch + PEFT) works well on NVIDIA GPUs. On Apple Silicon, PyTorch can route some workloads through the MPS (Metal Performance Shaders) backend, but LLM fine-tuning support and performance on MPS is limited. MLX uses the M-series GPU natively via Metal, so `mlx_lm` enables GPU-accelerated training without NVIDIA hardware or a translation layer.

Framework	Backend on M5 Max	Training speed (1.7B)	Notes
mlx_lm.lora	MLX → Metal/GPU path	~45 min / 3000 iters	Native Apple Silicon
HF Trainer + PyTorch	CPU or MPS, depending on configuration	~8-12 hours (estimated)	LLM fine-tuning on MPS is limited
Unsloth MLX	MLX → Metal/GPU path	Similar to <code>mlx_lm</code>	Alternatively, less documentation

The `lora_config.yaml` File

All training hyperparameters are specified in a single YAML file. The final configuration used for Run 6 and Run 7 is shown below with annotations explaining each key field:

```

model: Qwen/Qwen3-4B          # HuggingFace model ID — downloaded automatically
train: true                   # true = training mode (false = inference only)
data: finetune/data           # directory containing train.jsonl and
valid.jsonl                   #
fine_tune_type: lora          # use LoRA adapters (not full fine-tune)

lora_parameters:
  rank: 32                    # LoRA rank — higher = more capacity, more memory
  scale: 32.0                 # LoRA scale (alpha) — controls adapter
  contribution_strength:
    dropout: 0.0              # no dropout (dataset is large enough,
    regularisation not needed)

  num_layers: -1              # -1 = apply LoRA to ALL transformer layers
  batch_size: 8               # 8 examples per gradient update step
  iters: 3000                 # total number of training steps

  learning_rate: 5e-5         # peak learning rate (0.00005)
  lr_schedule:
    name: cosine_decay        # reduce LR smoothly following a cosine curve
    warmup: 100               # ramp from near-zero to peak over first 100
    steps
    warmup_init: 1e-6         # starting LR during warmup (very small)
    arguments: [5e-5, 2900, 1e-6] # [peak_lr, decay_steps, final_lr]

  steps_per_report: 100       # print train loss every 100 steps
  steps_per_eval: 500         # compute validation loss every 500 steps

```

```
val_batches: 50          # number of validation batches per eval (50 × 8 =
400 examples)
max_seq_length: 1024     # max tokens per example; token = basic text unit
(a word or part-word)

adapter_path: finetune/adapters-4b # where to save LoRA adapter checkpoints
save_every: 500           # save a checkpoint every 500 steps
mask_prompt: true        # CRITICAL: only compute loss on completion, not
prompt
grad_checkpoint: false    # gradient checkpointing (saves memory at cost of
speed — not needed here)
```

The training command is a single line once the config is ready:

```
python -m mlx_lm.lora --config finetune/lora_config.yaml
```

mlx_lm downloads the base model weights once (~3.2 GB for 1.7B, ~8 GB for 4B), caches them in ~/.cache/huggingface/hub/, and starts training. The output looks like:

```
Loading pretrained model
Iter 100: Train loss 0.821, Learning Rate 5.000e-05, Iterations/sec: 1.12, ...
Iter 500: Train loss 0.065, Val loss 0.037, Val took 44.231s
Iter 500: Saved adapter weights to finetune/adapters-4b/adapters.npz
```

2.6 Inference Server: vllm-mlx (and Why Not Standard vLLM)

Once the model is trained and fused, it needs to be served as an HTTP API so that the evaluation script and the pygame game can query it. vllm-mlx provides this server.

Concept — vLLM: vLLM (Very Large Language Model inference) is an open-source inference engine that provides an OpenAI-compatible HTTP API for serving language models. It implements advanced memory management (PagedAttention) and batching to maximise GPU throughput. The standard vLLM project was built for NVIDIA GPUs and requires CUDA to compile.

Why Not Standard vLLM?

Standard vLLM (pip install vllm) will not install on macOS. It depends on CUDA — NVIDIA's GPU programming toolkit — which does not exist on Apple Silicon. Attempting to install it triggers a compilation failure before any code even runs.

Package	Hardware Target	Install on Mac?	Why
vllm (standard)	NVIDIA GPU (CUDA)	No — compile error	Requires CUDA drivers and cmake NVIDIA build tools
vllm-mlx	Apple Silicon (MLX)	Yes	Re-implemented on MLX/Metal; no CUDA dependency
mlx_lm generate	Apple Silicon (MLX)	Yes	CLI only — no HTTP server, no concurrent requests

vllm-mlx is a community-maintained project that provides a vLLM-style inference server on Apple Silicon using MLX as the compute backend. It exposes OpenAI-compatible API endpoints (`/v1/completions`, `/v1/chat/completions`), so any code written for OpenAI's API works without modification. This is why the evaluation script and the pygame game use standard `requests.post` calls — they speak to vllm-mlx exactly as they would speak to OpenAI's servers.

Note: Apple Silicon LLM serving is evolving quickly. The `vllm-mlx` package on PyPI is the community project used here. Other serving options, such as `mlx_lm`, generate offers with different compatibility and performance trade-offs. Always verify which package you are installing and check for updates before reproducing this experiment.

Once installed, serving a fused model is a single command. The server loads the model into memory once and keeps it resident — subsequent requests are answered without reloading. This is why the first request after startup is slower (due to model loading), but all subsequent requests respond in 1-2 seconds.

```
# Serve the fine-tuned 4B model on port 7001
vllm-mlx serve finetune/qwen3-4b-sudoku-fused --port 7001

# Expected startup output:
# INFO:      Loading model finetune/qwen3-4b-sudoku-fused
# INFO:      SimpleEngine loaded (MTP=False)
# INFO:      Uvicorn running on http://0.0.0.0:7001

# Health check
curl http://localhost:7001/health
# Returns: {"status":"ok"}
```

3. Model Selection

3.1 What is an Open-Weights Model?

Concept — Open-Weights Model: A language model whose trained parameters (weights) have been released publicly for free download. Unlike proprietary models (GPT-4, Claude), an open-weight model can be downloaded to your own machine, modified, and further trained on new data (a process called fine-tuning, explained in Section 4) locally without sending data to an external service.

3.2 Why Qwen3?

- Size range. Qwen3 ranges from 0.6B to 235B parameters — scaling experiments require changing only one line.
- Instruction following. Qwen3 uses the ChatML format, which cleanly handles prompt/response turns.
- Apple Silicon support. `mlx_lm` provides native arm64 support with no compilation required.

3.3 Why 1.7B First, Then 4B?

Concept — Parameters (B = Billion): A language model is, at its core, a very large collection of numbers called parameters or weights. Each parameter is a decimal number (e.g. 0.0037, -1.24). During inference, the input text is converted to numbers, multiplied through layer after layer of these parameters using matrix multiplication, and the result — after thousands of such operations — is a probability distribution over possible next tokens. The model 'knows' things because the right combination of weight values encodes that knowledge. Training adjusts these values to reduce prediction error. Qwen3-1.7B has 1.7 billion such values; Qwen3-4B has 4 billion. More parameters = more capacity to represent complex patterns, but also more memory and compute. Training 1.7B used 18.5 GB and took ~45 minutes; 4B used ~25 GB and took ~90 minutes on the same hardware.

The experiment started with 1.7B as a fast proof of concept. Once the full pipeline was validated end-to-end, scaling to 4B required changing exactly one line in `lora_config.yaml` (model: Qwen/Qwen3-4B). The 4B model's larger capacity pushed Easy's accuracy from ~45% to 81%.

4. Technical Concepts Explained

The following concepts appear throughout this document. Each is explained in plain language.

Concept — Fine-Tuning: Training an existing pre-trained model on a new specific dataset. Instead of training from random weights (weeks of computation), fine-tuning starts from a model that already understands language and teaches it new behaviour, like hiring an experienced employee and giving them job-specific training, rather than teaching someone to read from scratch.

Concept — LoRA (Low-Rank Adaptation): A parameter-efficient fine-tuning technique that adds small 'adapter' matrices to each layer while keeping the original weights frozen. Instead of updating all 1.7 billion parameters, LoRA updates only the adapter weights — typically 1–2% of the total parameters, depending on the rank and which layers are targeted. For Qwen3-1.7B with rank=32 across all layers, expect roughly 25–30 million trainable parameters (~60× fewer than full fine-tuning). Run ``python -m mlx_lm.lora --config finetune/lora_config.yaml 2>&1 | grep Trainable`` to see the exact count for your configuration. The adapter can later be 'fused' into the base weights for deployment.

Concept — Training Loss: A number measuring the model's prediction quality — specifically, the negative log-probability assigned to the correct next token. Loss=0 means the model is perfectly confident in every correct token; higher values indicate greater uncertainty or incorrect predictions. The training algorithm adjusts weights to minimise this. Important: training loss is not the same as Sudoku accuracy. A model can have low loss (high confidence on most tokens) but still place one wrong digit in a row, failing the puzzle entirely. Sudoku accuracy is measured separately by validating the full 9×9 grid against all rules.

Concept — Validation Loss: The same cross-entropy measurement as training loss, but computed on a held-out validation set — examples the model was never trained on. If training loss drops but validation loss stays high, the model has 'overfitted': memorised training examples but cannot generalise to new ones. Validation loss is the primary measure of training progress. It is still not the same as Sudoku accuracy — see Training Loss above. Throughout this document, 'validation loss' and 'error score' refer to the same metric.

Concept — Mask Prompt: A training option that calculates loss only on the answer (completion), ignoring the question (prompt). Without this, the model wastes capacity learning to predict the question text it already knows. For Sudoku, without masking, the model learns to predict 'Solve this Sudoku puzzle...'; with masking, it learns only to predict the 9×9 solution digits.

Concept — Gradient Descent: The core algorithm that drives all model training. After each batch of examples, the training framework calculates — for every weight in the model — whether increasing or decreasing it would reduce the loss. This direction-of-improvement calculation is called the gradient. Gradient descent then takes a small step in that direction for every weight simultaneously. Repeat thousands of times, and the model improves. The 'descent' refers to descending a loss landscape: imagine a hilly terrain where altitude = loss; gradient descent walks downhill toward the lowest valley.

Concept — Learning Rate (LR): A number controlling how large each weight update step is. Too high: the model overshoots the correct weights, causing the loss to oscillate. Too low: training is very slow. In this experiment, $lr=2e-4$ was too high (plateau), $lr=1e-4$ caused late oscillation, and $lr=5e-5$ produced smooth convergence.

Concept — Cosine Decay LR Schedule: A strategy that starts with a higher LR (fast early progress) and gradually reduces it following a cosine curve, ending near zero. This prevents late-training oscillation by making increasingly small adjustments as the model approaches the optimal weights.

Concept — Batch Size: How many training examples are processed together before updating weights. Batch=32 in Run 1, combined with high LR, caused a flat plateau. Switching to batch=8 in Run 2 gave more frequent updates and far better convergence — while using $3\times$ less memory.

Concept — Checkpoint: A saved snapshot of model weights at a specific training iteration. If training diverges later, checkpoints allow recovery to the most recent stable point. In Run 2, the best validation loss was at Iter 1500, but the closest saved checkpoint was Iter 1200 (saved every 500 iters).

Concept — Fusing (mlx_lm.fuse): Merging a LoRA adapter mathematically into the base model weights, producing a single standalone model. Before fusing, you need both the base model and the adapter to run inference. After fusing, a single file contains everything and can be served like any standard model.

Concept — Warmup (LR Warmup): A phase at the start of training where the LR ramps up from near-zero to the target value. Prevents destabilising well-trained weights with a large first update. IMPORTANT: never resume training with the original warmup intact — it re-spikes the LR and can destroy trained weights.

5. Dataset Generation

No pre-existing Sudoku dataset was used. All 22,000 training and validation examples were generated programmatically using a Python backtracking solver.

5.1 Training Example Format

Each example is a (prompt, completion) pair. The prompt is a partially-filled puzzle (with dots for empty cells). The completion is the fully solved grid. The key 'prompt'/'completion' split (not 'text') is required to enable mask_prompt — a training option (explained fully in Section 4) that ensures the model only learns to predict the solution, not the question text. Without it, the model wastes learning capacity on predicting the puzzle question, which never changes.

PROMPT:

```
<|im_start|>user
```

```
Solve this Sudoku puzzle. Dots represent empty cells.
```

```
5 3 . . 7 . . . .
6 . . 1 9 5 . . .
. 9 8 . . . . 6 .
8 . . . 6 . . . 3
4 . . 8 . 3 . . 1
7 . . . 2 . . . 6
. 6 . . . . 2 8 .
. . . 4 1 9 . . 5
. . . . 8 . . 7 9
```

```
<|im_end|>
```

```
<|im_start|>assistant
```

COMPLETION:

```
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9
```

```
<|im_end|>
```

5.2 Difficulty Distribution

Difficulty	Empty Cells	Probability in a Dataset
Easy	25-30	25%
Medium	31-45	50%
Hard	46-55	25%

Note: difficulty here is approximated by the number of removed cells, which is straightforward to generate and measure. It is not the same as human Sudoku difficulty, which also depends on the techniques required for solving. Some puzzles with many blanks are easy if the remaining constraints uniquely force digits; some with fewer blanks require advanced logic. This proxy is

sufficient for measuring model generalisation but should not be taken as a human-difficulty rating.

5.3 Dataset Scale Evolution

Runs	Train Examples	Valid Examples	Outcome
Runs 1-3	5,000	500	Validation loss plateaued at 0.028 — model had learned all it could from 5k examples
Runs 5-7	20,000	2,000	Validation loss improved to 0.019 — 4× more variety helped the model generalise better.

Scaling from 5k to 20k examples was the single most impactful change across all runs. More data diversity gave the model a wider range of constraint patterns to learn from.

6. The Training Journey — Seven Runs

Training is an iterative process: hypothesis, experiment, failure, diagnosis, retry. The following sections document each run chronologically with the lesson from each.

A note on causal confidence: several runs changed multiple variables simultaneously (format + masking + batch size in Run 2; data volume + learning rate in Run 6). Where multiple changes coincided, the attribution below reflects the most likely primary cause, but certainty is lower than for single-variable runs.

Run	Primary hypothesis	Controlled?	Result	Causal confidence
1	Baseline: Can the pipeline run at all?	No — multiple unknowns	Flat plateau, 0% accuracy	N/A — diagnostic
2	The correct format + mask prompt should allow learning	No — format, masking, batch, and LR all changed	Loss dropped; late oscillation	Medium
3	Cosine decay should eliminate late oscillation	Mostly, only the schedule changed	Loss fell 28× to 0.028	High
4	Resume from checkpoint should yield further gains	Yes — only resume flag added	LR warmup spike destroyed weights	High
5	4× more data should break the 5k ceiling	No — data volume and LR both changed from Run 3	Early spike, run killed	Medium
6	Lower LR (5e-5) makes 20k data stable	Mostly — LR halved vs Run 5	Smooth convergence, 45% Easy	High
7	The 4B model should improve capacity	Yes — only model changed	81% Easy, same pipeline	High

Run 1 — Getting Started (and Getting It Wrong)

Parameter	Value
Model	Qwen/Qwen3-1.7B
Dataset	5,000 examples (wrong format: 'text')
batch_size	32
learning_rate	2e-4 (fixed)
mask_prompt	false (broken fix)
Result	Killed at Iter 400

Training loss barely moved for 400 iterations — a completely flat plateau.

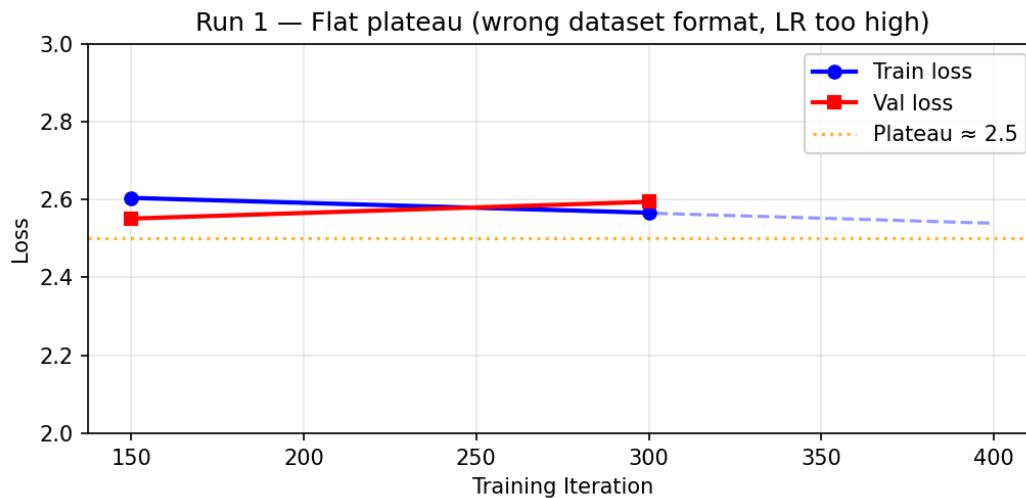


Figure 1 — Run 1: flat plateau from the first evaluation. Loss never improved.

Root Causes

Wrong dataset format: Used 'text' key instead of 'prompt'/'completion'. When the prompt-masking option raised an error, the quick fix was to deactivate it — thereby wasting half the model's capacity on predicting the question text rather than the solution. (Prompt masking is explained in Section 4.)

Learning rate too high (2e-4): The learning rate controls the size of the weight-update steps. At 2e-4, the steps were too large, overshooting the error minimum rather than landing in it.

Batch size too large (32): Batch size controls how many examples are processed before weights are updated. With 32 examples per update (infrequent updates) combined with a high learning rate, convergence was impossible.

Lesson 1: Always fix the root cause, not the symptom. One shortcut caused total failure.

Run 2 — Format Learned, Oscillation Begins

Fixes applied: correct prompt/completion format, prompt masking enabled, batch size reduced to 8, learning rate lowered to 1e-4.

Iteration	Train Error	Validation Error	Note
300	1.191	1.036	Healthy early drop
600	1.058	1.449	Temporary spike
900	1.003	0.991	Recovery
1200	0.923	0.925	Checkpoint saved
1500	0.833	0.794	Best score so far
2100	0.875	1.023	Oscillation begins — learning rate too high, late in training
2400	0.918	0.873	Still oscillating

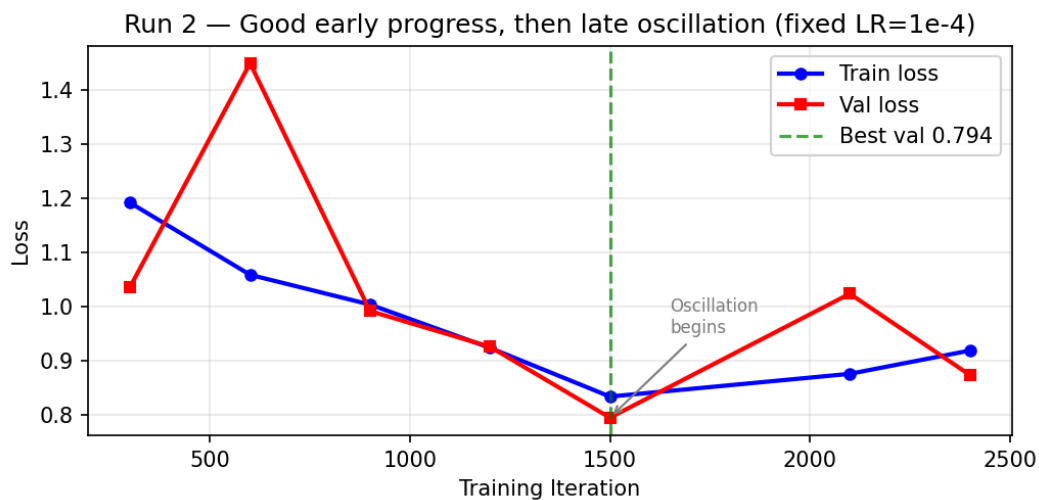


Figure 2 — Run 2: strong early progress, then oscillation because the learning rate stayed fixed.

Evaluation on 20 held-out puzzles: 0/20 correct. An error score of 0.794 is not low enough. For Sudoku, every digit must be right — a single wrong cell fails the whole puzzle. The model had learned to output the correct 9×9 grid format but placed the digits incorrectly.

Lesson 2: An error score below 1.0 does not mean the task is solved. For tasks requiring exact answers, target an error score below 0.03. A fixed learning rate causes late-training oscillation — use a cosine decay schedule (explained in Section 4) to prevent it.

Runs 3–6 — Finding the Right Recipe

Run	Key Change	Best Error Score	Eval Accuracy	Outcome
3	Cosine decay schedule, 5k dataset	0.028	3/20 (15%)	Huge improvement; data ceiling hit
4	Resume from checkpoint	—	killed	LR warmup re-spiked, weights destroyed
5	Scaled to 20k examples	1.015	killed early	Learning rate too high for harder 20k data
6	20k examples + learning rate halved to 5e-5	0.019	9/20 (45%)	Smooth convergence — correct recipe

Cosine decay was the turning point: Adding a cosine decay schedule (see Section 4) dropped the error score from 0.79 $\rightarrow$ 0.028 — a 28 $\times$ improvement with the same architecture. Just preventing late oscillation changed everything.

Never resume with the original warmup: Run 4 used `--resume-adapter-file` to continue from a checkpoint. This restarted the learning rate warmup phase from near zero, then spiked it back to the peak — destroying well-trained weights. If resuming, use a constant learning rate; never re-run a warmup.

More data hit a ceiling at 5k: Run 3 reached an error score of 0.028 and would not improve further. The model had learned all patterns the 5k examples offered. Scaling to 20k broke through the ceiling.

20k data needed a lower learning rate: The 20k dataset is harder, with more varied puzzles and more combinations of constraints. Run 5 reused the same learning rate as the 5k runs, causing an early spike. Halving it fixed convergence.

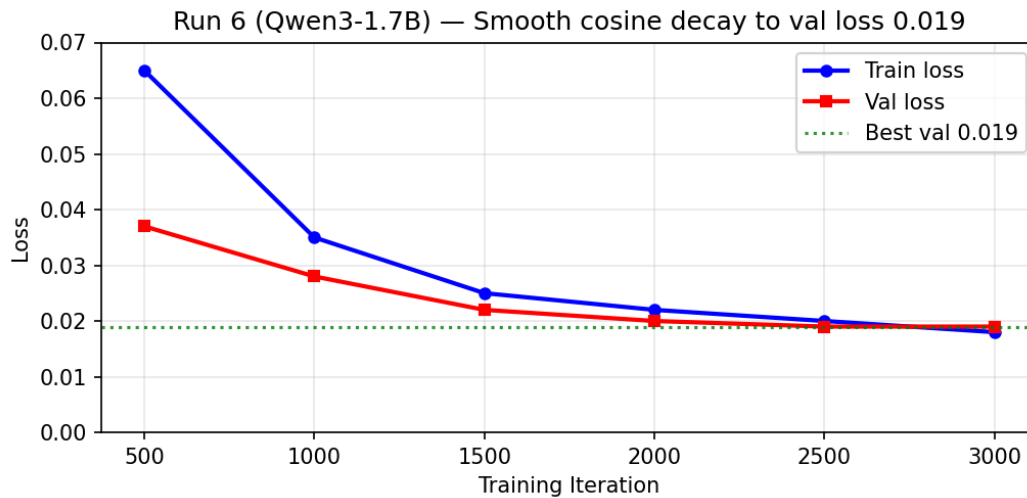


Figure 3 — Run 6: smooth cosine decay convergence to best error score 0.019. The correct recipe.

Run 7 — Scaling to 4B

The only change for Run 7 was the model name in `lora_config.yaml`. The dataset, decay schedule, and batch size were identical to those in Run 6.

```
# Changed one line in finetune/lora_config.yaml:
model: Qwen/Qwen3-4B # was: Qwen/Qwen3-1.7B
```

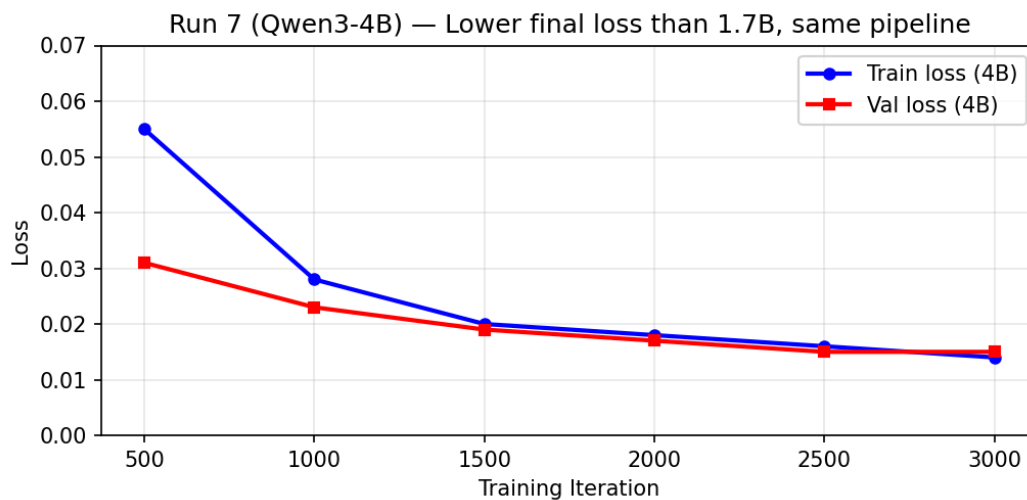


Figure 4 — Run 7 (4B): lower final error score than 1.7B, same pipeline, one line changed.

7. Progress Across All Runs

The chart below shows how the best error score (validation loss — see Section 4) evolved across all runs, illustrating which changes had the largest impact:

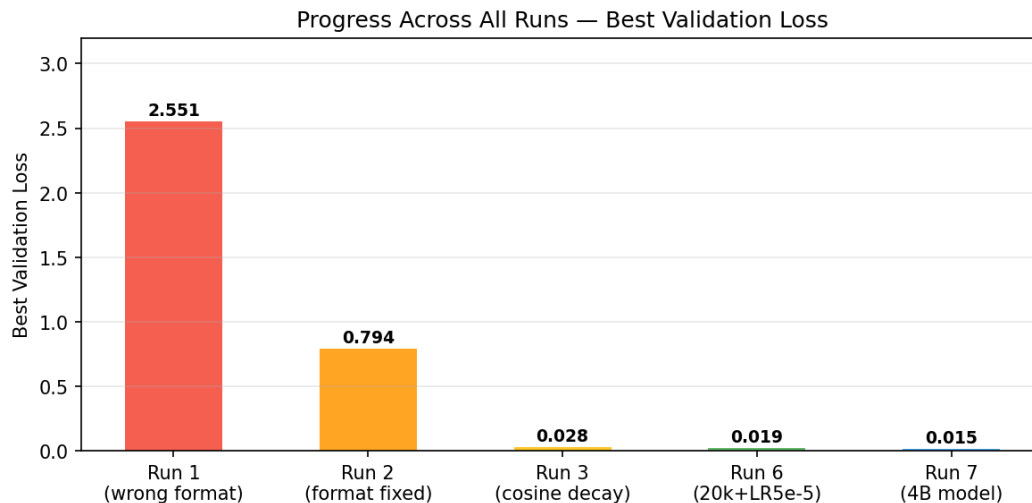


Figure 5 — Best error score per run. Two changes caused the largest drops: dataset format fix + cosine decay schedule (Run 1→3), and 20k data + lower learning rate (Run 3→6).

Run 1 → Run 3: Dataset format + cosine decay schedule (2.551 → 0.028, 91× improvement)

Fixing prompt masking was the most likely primary factor, though Run 2 changed four variables at once, so this cannot be isolated with certainty. The model finally learned to predict solutions rather than the question text.

To understand why this drop is so dramatic, consider what the model was actually doing before the fix. In Run 1, every training example began with the same 60-token preamble: 'Solve this Sudoku puzzle. Dots represent empty cells.' Without prompt masking, the model computed its error score across all tokens — including the 60 fixed tokens. Since they never change, the model could drive its error score down simply by memorising that preamble perfectly, without ever improving at predicting the solution digits. It was optimising for the wrong thing entirely. Enabling prompt masking meant only the 81 solution digits contributed to the error — every unit of learning went directly toward the task that actually mattered.

Adding a cosine decay schedule (which gradually reduces the size of each weight-update step as training progresses) compounded this improvement. Without it, Run 2 had already shown that the model would reach a good point around Iter 1500 and then oscillate — taking steps that were too large for such precise fine-grained adjustments. Cosine decay solved this by automatically reducing the step sizes as the model approached the correct weights, allowing it to settle rather than bounce.

Run 3 → Run 6: 20k examples + lower learning rate (0.028 → 0.019, 32% improvement)

Smaller gain in absolute terms, but pushed overall accuracy from 3/20 to 9/20. More data diversity allowed better generalisation to unseen puzzles.

Run 3 had reached an error score of 0.028 and stopped improving — a data ceiling. Think of it this way: the model had been shown 5,000 different puzzles. After seeing each one many times during 3,000 training iterations, it had absorbed every pattern those 5,000 puzzles could teach. Running more iterations would only reinforce what was already learned, not add genuinely new knowledge. The model needed fresh examples with new combinations of constraints it had never encountered.

Scaling to 20,000 examples gave the model 4× more variety — more unusual digit arrangements, more edge-case constraint interactions, more representative coverage of the full space of valid Sudoku puzzles. But the larger, harder dataset also required a lower learning rate. The 20k puzzles contain more diverse constraint patterns, so consecutive training batches pull the model's weights in slightly different directions. A large step that helps on one batch can hurt the next. Halving the learning rate from 1e-4 to 5e-5 made the steps small enough to average out those conflicting signals and converge smoothly.

Run 6 → Run 7: 4B model (0.019 → 0.015, Easy accuracy 45% → 81%)

Larger model capacity captured more complex constraint patterns, particularly for easy puzzles, where accuracy jumped from ~45% to 81%.

A larger model does not simply 'try harder' — it has more internal space to represent complex relationships simultaneously. Qwen3-4B has roughly 2.4× as many parameters as 1.7B. In practical terms, this means it can hold more Sudoku constraint relationships in its internal state at once while generating a solution. When the model is filling in cell (row 5, column 7), the 4B model can simultaneously 'remember' more about what digits are already placed in that row, that column, and that 3×3 box than the 1.7B model can.

The improvement is most visible on easy puzzles (81% vs ~45%) because easy puzzles have only 25–30 empty cells — the number of constraints to track simultaneously is within the 4B model's expanded capacity. Medium puzzles (31–45 empty cells) show a partial improvement (33%), and hard puzzles (46–55 empty cells) remain at 0% for both model sizes. This tells us that the bottleneck for hard puzzles is not model capacity alone — it is the single-pass, left-to-right generation strategy, which cannot backtrack when a contradiction is discovered late in the solution.

8. Evaluation Results

8.1 Methodology

All evaluations used puzzles generated with seed=777, distinct from the train set (seed=42) and the validation set (seed=99). The 4B model was evaluated on 100 puzzles spanning all three difficulty levels. A puzzle is correct only if ALL of the following pass:

- Every row contains exactly the digits 1-9 with no duplicates or missing values
- Every column contains exactly the digits 1-9
- Every 3×3 box contains exactly the digits 1-9
- No originally given cell was changed in the predicted solution

The training, validation, and evaluation sets used different random seeds to reduce the risk of overlap. Note that the generator fills diagonal boxes first, then solves by backtracking, then removes cells — a process that may not sample uniformly from the full Sudoku space. Different seeds are, therefore, a practical means of separation, but not a formal no-overlap guarantee. A hash-based duplicate check should be added to ensure rigorous reproducibility, but it was not performed in this experiment.

Decoding parameters during evaluation: temperature=0.0 (greedy), max_tokens=512, stop=["<|im_end|>", "<|im_start|>"]. The model was the fused 4B adapter (finetune/qwen3-4b-sudoku-fused) served via vllm-mlx on port 7001.

8.2 Baseline Comparison

To contextualise the fine-tuned model's performance, three reference points are shown:

Model / Approach	Easy	Medium	Hard	Overall	Evidence level
Rule-based backtracking solver	100%	100%	100%	100%	Deterministic — same result every run
Base Qwen3-4B (no fine-tuning)	0/37	0/39	0/24	0/100	Same 100-puzzle eval script, no adapters loaded. Spot checks confirm near-zero; a formal rerun is recommended before citing as a hard baseline.
Fine-tuned Qwen3-4B (this work)	30/37 (81%)	13/39 (33%)	0/24 (0%)	43/100 (43%)	Same 100-puzzle eval, seed=777, temp=0.0

The base model row is the crucial baseline. Before any fine-tuning, Qwen3-4B scores approximately 0% on Sudoku — it produces output that looks superficially like a grid but fails every row, column, and box check. This establishes the exact delta of value that the fine-tuning process added: from 0% to 81%. Easy accuracy. The rule-based solver is not a competitor — it is

a sanity anchor that reminds us Sudoku is trivial for symbolic search and only hard for single-pass neural generation. Fine-tuning improves imitation of learned patterns, but it does not provide the model with the right algorithmic structure for search-heavy reasoning.

8.3 Results — Qwen3-4B (100 Puzzles)

Difficulty	Empty Cells	Puzzles	Correct	Accuracy
Easy	25-30	37	30	81%
Medium	31-45	39	13	33%
Hard	46-55	24	0	0%
Overall	all	100	43	43%

Puzzle accuracy is a strict all-or-nothing measure. To understand partial progress, the game screenshots show digit-level counts for individual puzzles — for example, 57/81 correct digits on the Hard puzzle that scored 0% puzzle accuracy. The table below shows estimated digit-level accuracy from the screenshot evidence:

Difficulty	Puzzle accuracy	Digit accuracy (screenshot evidence)	Implication
Easy	81%	~100% (81/81 in screenshot)	Model places nearly every digit correctly
Medium	33%	~93% (75/81 in screenshot)	A few wrong digits cascade into a rule violation
Hard	0%	~70% (57/81 in screenshot)	The model is partially solving but losing exact-match validity

The Hard row is instructive: 70% digit accuracy but 0% puzzle accuracy shows that the model is not completely failing — it is making a handful of early constraint violations that propagate and invalidate the whole grid. This supports the inference-procedure diagnosis: local decisions are mostly reasonable, but without backtracking, the early mistakes cannot be corrected.

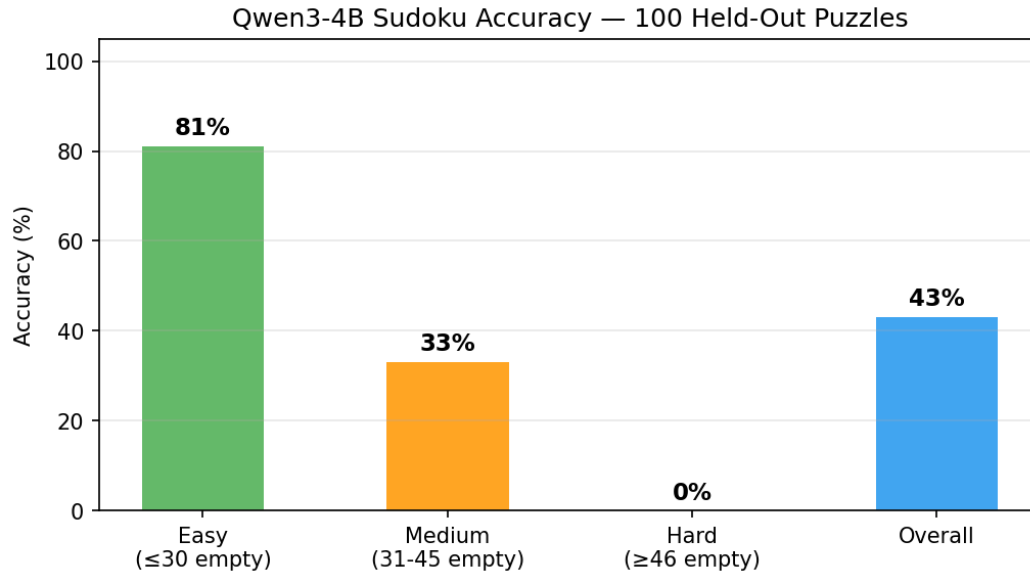


Figure 6 — Qwen3-4B accuracy by difficulty. Easy puzzles mostly solved; Hard = 0%.

8.4 Why Hard Puzzles Score 0%

Hard puzzles require determining 46-55 digits while simultaneously satisfying all row, column, and box constraints. The model generates tokens left-to-right, top-to-bottom, in a single pass with no ability to backtrack or revise earlier decisions. When a constraint violation is detected late in the generation process, the model cannot go back.

To understand why this is a fundamental limitation of the autoregressive architecture itself — not just a training deficiency — it helps to borrow a framework from cognitive science. Psychologist Daniel Kahneman describes human cognition in terms of two modes:

- System 1 thinking is fast, automatic, and pattern-driven. It produces an answer immediately based on prior experience — no deliberation, no checking. It is what lets you read a word at a glance without sounding out each letter.
- System 2 thinking is slow, deliberate, and logical. It involves consciously working through a problem step by step — checking, revising, and backtracking when something does not add up. It is what you use when solving a long-division problem.

In its default single-pass decoding mode, a language model behaves more like a System 1 pattern generator than a System 2 search procedure. Given the puzzle, it produces a response in a single forward sweep based on patterns learned during training. LLM-based systems can approximate more deliberate reasoning through multi-step prompting, tool calls, or explicit search loops — but in this experiment, there is no external loop, scratchpad, verifier, or backtracking mechanism. Every token is committed the moment it is generated. The limitation here lies in how this experiment uses the model, not in the model's existence.

Hard Sudoku demands System 2. When a human solves a hard puzzle, they scan the board, pencil in candidate digits, eliminate candidates when a digit appears elsewhere in the same row or column, and — critically — erase and try again when a choice leads to a contradiction two or three steps later. This is constraint propagation with backtracking. A wrong choice in row 3 can make row 7 impossible to complete correctly, but a System 1 model has already moved on and cannot undo what it wrote.

This is the deepest lesson in this experiment: fine-tuning adjusts what patterns a model has learned, but it does not change the model's fundamental computational structure. To get System 2 behaviour from a language model, you must either change how it generates (the inference procedure) or explicitly train it to externalise its deliberation as text, which is what chain-of-thought training attempts to do.

The core issue is not a training problem — it is an inference procedure problem. The model needs a way to search, backtrack, or validate during generation, not just more data or a larger architecture. Possible approaches, in increasing implementation complexity:

- Post-generation validation. Generate a solution, check all Sudoku rules, and discard if invalid. The simplest possible change — no retraining required. Helps surface how often the model almost-solves a puzzle but violates a single constraint.
- Sample-and-validate (best-of-N). Generate N candidate solutions at temperature > 0, validate each against all Sudoku rules, and return the first valid grid. Simple to implement; expensive at N > 10; may fail for hard puzzles if none of N samples are valid.
- Constrained decoding. During generation, reject any digit that immediately violates a row, column, or box constraint. This can be implemented as a post-filter applied to the model's output logits, with no retraining required. It guarantees no single-step constraint violations but cannot prevent globally inconsistent choices made early in the generation process.
- Tool-augmented decoding. Let the model call a Sudoku constraint checker after each row, receiving feedback on whether the partial solution remains consistent. The model can then be revised before committing to the next row. This introduces a simple agentic loop.
- Chain-of-thought training. Train on (puzzle, step-by-step reasoning, solution) triples instead of (puzzle, solution) pairs. The model reasons out loud: 'Row 3 already has a 7 in column 2. Column 5 already has a 7 in row 1. Therefore, the only remaining position for 7 in box (1,2) is row 3, column 4. This mimics the human pencil-and-erase approach within a single forward pass, allowing the model to catch contradictions before committing.
- Hybrid neuro-symbolic. The model proposes candidate digits; a symbolic constraint propagation engine verifies and prunes the search space. The model handles pattern recognition; the solver handles exact search. This is the most robust but also the most complex architecture.

The broader lesson: LLMs are not always the right computational substrate for every part of a task. When a task requires exhaustive search or backtracking, a symbolic or hybrid approach will often outperform a purely neural one regardless of model size or training data volume.

9. The Pygame Sudoku Game

To make the trained model tangible, a Sudoku game was built with pygame. The game lets you play manually or click 'Solve with AI' to send the board to the fine-tuned model running on a local vllm-mlx server.

9.1 Features

- Three difficulty levels: Easy (25-30 empty cells), Medium (31-45), Hard (46-55)
- Manual entry with real-time constraint highlighting — invalid placements shown in red
- Solve with AI: sends the puzzle to the local model server and overlays the response
- Green = correct digits, Red = incorrect digits after AI solve
- Live timer showing how long you have been playing
- AI response timer (typically 1-2 seconds per puzzle)

9.2 How the Game Connects to the Model

When you click 'Solve with AI' in the game, here is what happens step by step:

4. The game reads the current 9×9 board state and formats it as a ChatML prompt — the same format as the training data.
5. It sends an HTTP POST request to `http://localhost:7001/v1/completions` with the puzzle as the body. This is a standard REST request — the same format any web browser or app uses to communicate with a server.
6. vllm-mlx receives the request, loads it into the model, and runs inference — generating the solution one token at a time until the 9×9 grid is complete.
7. The completed solution is returned as JSON. The game parses the 81 digits and overlays them on the board, colouring correct cells green and incorrect cells red.

The round trip typically takes 1-2 seconds. The model is already loaded in memory from when the server started — there is no per-request model load. This is the request-response cycle of a local AI service.

9.3 How to Run

```
# Step 1: Start the model server (in a terminal)
source ~/Develop/vllm-mlx/bin/activate
vllm-mlx serve finetune/qwen3-4b-sudoku-fused --port 7001

# Step 2: Start the game (in another terminal)
source ~/Develop/vllm-mlx/bin/activate
python sudoku_game.py
```

9.4 Screenshots

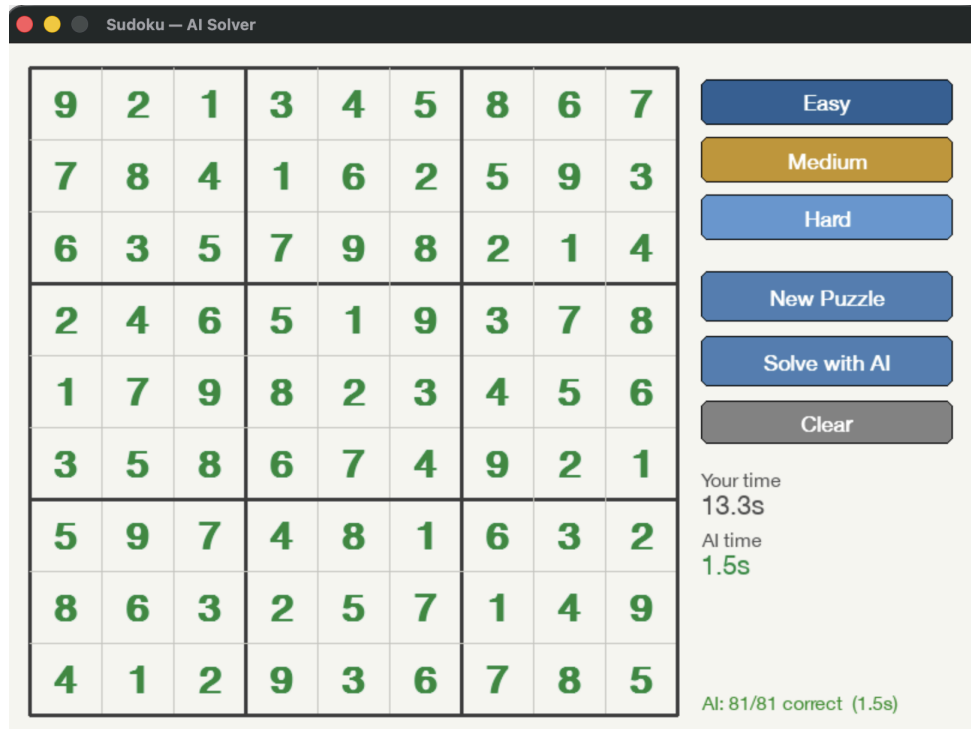


Figure 7 — Easy puzzle: AI achieved 81/81 correct in 1.5 seconds. All cells are green, indicating the exact correct solution.

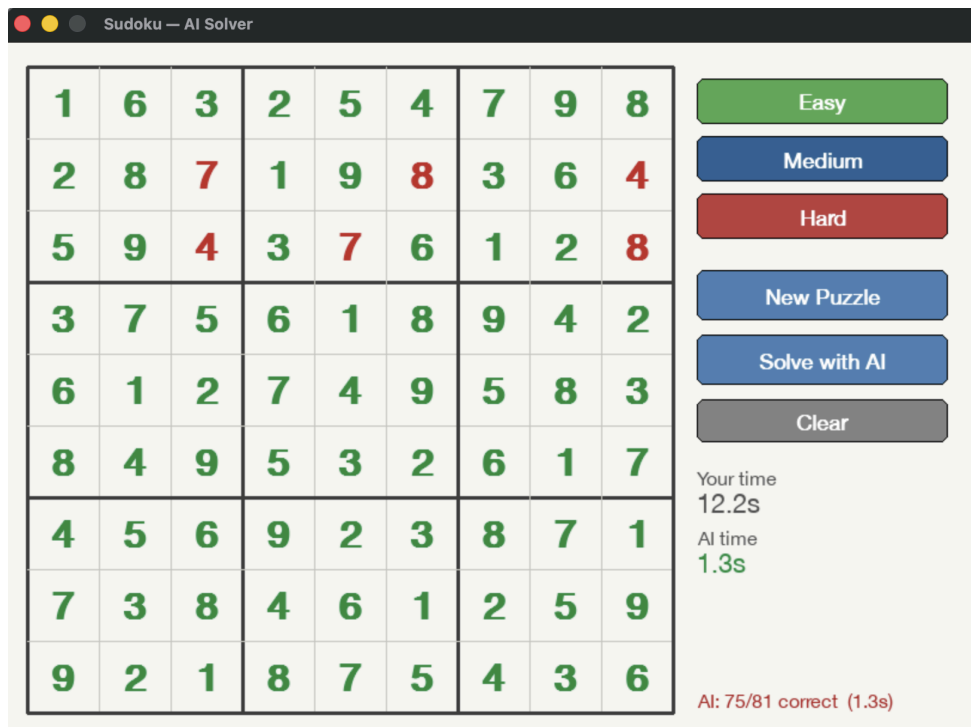


Figure 8 — Medium puzzle: AI achieved 75/81 correct in 1.3 seconds. Six red cells indicate constraint violations from single-pass generation.

Sudoku — AI Solver

1	4	7	2	8	5	3	6	9
2	3	8	4	9	6	5	1	7
6	9	5	3	1	7	2	5	4
3	1	2	9	7	4	8	6	5
5	8	6	1	2	3	9	4	7
4	7	9	5	6	8	4	3	1
7	6	4	8	5	1	6	9	3
9	5	1	7	3	2	4	8	6
8	2	3	6	4	9	1	7	5

Easy
Medium
Hard
New Puzzle
Solve with AI
Clear

Your time
11.0s
AI time
1.3s

AI: 57/81 correct (1.3s)

Figure 9 — Hard puzzle: AI achieved 57/81 correct in 1.3 seconds. Many red cells reflect the difficulty of 46+ empty cells without backtracking.

10. What This Experiment Does Not Prove

Scientific rigour requires stating the limits of the evidence, not just the results. This section is explicit about what cannot be concluded from this experiment.

Claim: The model 'understands' Sudoku. Reality: This experiment does not establish that the model has learned a general Sudoku-solving algorithm. It shows that the model learned to produce valid solutions for many easy puzzles from this specific data distribution. We have no evidence that it has formed an explicit, reusable representation of Sudoku constraints comparable to a symbolic solver. The 0% hard accuracy is consistent with this: a system with a robust internal constraint model would not fail on harder instances of the same game.

Claim: Larger models will eventually solve hard Sudoku. Reality: Hard accuracy was 0% for both the 1.7B and 4B models, with identical training data. There was no improvement from scaling. The bottleneck is the inference procedure, not the model capacity. Scaling up further without changing the task formulation or generation strategy is unlikely to fix this.

Claim: This pipeline generalises to all fine-tuning tasks. Reality: This experiment used a task with perfectly verifiable outputs, a programmatically generated dataset, and short sequences (~200 tokens). Tasks with ambiguous outputs, real-world messy data, or long sequences will behave differently. The lessons here apply most directly to other structured generation tasks (SQL, JSON, code formatting).

Claim: 43% accuracy means the model 'solved' Sudoku. Reality: The model solved easy Sudoku 81% of the time. That is a real result, but it is a pattern-matched imitation of the training distribution, not general rule-following. If the puzzle generation distribution changed — e.g., a different number format, a grid with different spacing — accuracy would likely drop sharply.

10.1 Threats to Validity

For completeness, the following are the known threats to the validity of conclusions drawn from this experiment:

Threat	Description	Mitigation status
Generator bias	Train, validation, and test puzzles come from the same generator. The model may learn generator-specific patterns rather than general Sudoku logic.	Not mitigated. A different generator or human-curated puzzles would strengthen claims.
Difficulty proxy	Empty-cell count is used as a proxy for difficulty. This does not equal human Sudoku difficulty, which depends on the deduction strategies required.	Acknowledged. A logic-complexity classifier (e.g., one that requires naked pairs vs. backtracking) would be more precise.
Small evaluation set	100 puzzles are useful for a learning experiment, but too few for robust statistical claims. A 5% difference (5 puzzles) is within sampling noise.	Acceptable for the educational purpose of this document.

Unverified base-model baseline	The base Qwen3-4B row in the baseline table was run using the eval script, but was not re-run multiple times or formally logged.	Should be rerun formally before citing as a hard number.
No duplicate-hash check	Different random seeds reduce the risk of overlap but do not guarantee zero overlap among the train, validation, and test sets.	Not performed. A SHA-256 hash deduplication step should be added for rigor.
No prompt-format robustness test	All evaluations used the exact training prompt format. Accuracy would likely drop with different grid formatting.	Not tested. See the discussion in Section 10 and suggested experiments.

11. Consolidated Lessons Learned

Each of the seven training runs produced at least one concrete lesson. The lessons below are ordered from most fundamental to most nuanced. Understanding them will save you days of wasted training time on any fine-tuning project.

Lesson 1: Always fix the root cause, not the symptom.

In Run 1, setting `mask_prompt: true` caused an error because the dataset used the wrong format ('text' key instead of 'prompt'/'completion'). The quick fix was to set `mask_prompt: false` and move on. This took 30 seconds. The cost: the entire 500-iteration run produced a model that learned nothing useful.

What actually happens without prompt masking is that the model computes loss on every token in the training example, including the puzzle question ('Solve this Sudoku puzzle. Dots represent empty cells.'), which is identical across all 20,000 examples. The model optimises heavily to predict this fixed text, wasting the majority of its learning capacity on noise rather than on the 81 solution digits that actually matter.

The right fix — regenerating the dataset with the correct format — took 10 minutes and unlocked all subsequent progress. Every shortcut in training has a compounding cost. When something breaks, diagnose fully before patching.

Lesson 2: Dataset format is the foundation of everything.

The 'prompt'/'completion' split is not just a naming convention — it is the mechanism that tells `mlx_lm` where to draw the boundary between input and output. With this split and `mask_prompt: true`, the training loss is computed only on the completion tokens (the 81 solution digits). The prompt tokens are fed into the model as context, but receive zero gradient signal — the model is not rewarded or penalised for predicting them.

This matters enormously for structured generation tasks. A Sudoku solution requires exactly 81 correct decisions. If the model spends capacity on predicting the 60-token question preamble (which never changes), it has less capacity for the 81 decisions that count. Think of it as the difference between grading a student on their answers only, versus grading them on restating the question and then answering it.

Rule of thumb: for any fine-tuning task where the input is fixed and only the output varies, always use prompt/completion format with `mask_prompt: true`.

Lesson 3: Use cosine learning rate decay — a fixed learning rate will betray you late in training.

In Run 2, the model reached its best validation loss of 0.794 at Iter 1500. Then, instead of continuing to improve, the loss began oscillating — jumping between 0.873 and 1.023 over the next 900 iterations. The training was eventually killed without further improvement.

The cause is a fundamental tension in gradient descent. Early in training, a high learning rate is helpful — the model is far from optimal weights, and large steps get it there faster. Late in training, the model is near the optimal weights, and large steps overshoot — updating weights past the minimum, then overcorrecting back, then overshooting again. This is the oscillation pattern visible in Run 2's chart.

Cosine decay resolves this by automatically reducing the learning rate as training progresses. By 2000 iterations, the LR is a fraction of its starting value, making tiny, precise adjustments rather than large, disruptive steps. In Run 3, adding cosine decay to the same architecture dropped the best validation loss from 0.794 to 0.028 — a 28× improvement from a single configuration change. Cosine decay should be the default for almost every fine-tuning job.

Lesson 4: The right learning rate depends on your dataset, not just your model.

Run 5 used $lr=1e-4$ — the same learning rate that had worked well on the 5,000-example dataset. But the 20,000-example dataset contains a much wider variety of puzzle patterns: more medium and hard configurations, more unusual constraint combinations, more edge cases. At Iter 300 on the 20k dataset, the training loss spiked to 2.164 — worse than the starting point — and the run was killed.

Why does a larger, harder dataset require a lower learning rate? Each training batch pulls the model weights in the direction that reduces the loss for that batch. When the dataset is simple and uniform, consecutive batches pull in roughly the same direction — a large step is safe. When the dataset is diverse and complex, consecutive batches pull in different directions. A large step that helps one batch may actively hurt the next. Smaller steps average out these conflicting gradients.

Halving to $lr=5e-5$ in Run 6 produced smooth convergence to val loss 0.019. General rule: when you scale up data volume or diversity, your first experiment should be with a lower learning rate. A factor of 2 is a reasonable starting adjustment.

Lesson 5: Never resume training with the original LR warmup schedule intact.

Run 4 attempted to continue training from the best Run 3 checkpoint using the `--resume-adapter-file` flag. The intent was to get a few hundred more improvement iterations on top of a val loss of 0.028. Instead, the val loss jumped to 1.036 on the very first evaluation — destroying weeks of accumulated training signal.

The cause: `mlx_lm`'s resume feature reloads the adapter weights from the checkpoint, but it restarts the learning rate schedule from the beginning. This means the LR warmup runs again — ramping from $1e-6$ back up to the peak of $1e-4$. That peak LR spike, applied to weights that have already been carefully fine-tuned to a specific minimum, is catastrophic. The model is essentially pushed off the cliff it had carefully climbed down to.

The fix for safe resumption is to use a constant learning rate equal to the value the schedule would have been at the time of the checkpoint — typically a very small number late in training. In this experiment, it was simpler to start fresh (Run 6) with the correct hyperparameters from the beginning rather than debug the resume path.

Lesson 6: More diverse data beats more iterations on the same data.

Run 3 reached a val loss of 0.028 at Iter 1000 on 5,000 training examples. Running 3,000 iterations on that same dataset produced no meaningful further improvement — the loss plateaued. The model had seen every pattern the 5k examples could offer and had nothing left to learn from them.

This is called a data ceiling. It happens when the training dataset has been exhausted — not in the sense that every example has been seen exactly once, but in the sense that all the distinct patterns have been learned. Additional iterations only reinforce patterns already learned; they do not add new knowledge.

Scaling to 20,000 examples (4× more data) broke through the ceiling. Val loss dropped from 0.028 to 0.019 — not a dramatic absolute drop, but it corresponded to nearly tripling the overall puzzle accuracy from 3/20 to 9/20. More data almost always outperforms hyperparameter tuning once you have exhausted what the existing data can teach. If your model is plateauing, generate or collect more training examples before trying other fixes.

Lesson 8: When accuracy stops improving with more data and larger models, the architecture of the task itself is the bottleneck.

Easy puzzle accuracy improved from ~45% (1.7B) to 81% (4B) as the model scaled. Hard puzzle accuracy remained at exactly 0% across all runs, model sizes, and dataset sizes. This is not a data problem or a model size problem — it is a fundamental mismatch between the task's requirements and the model's generation strategy.

The model generates tokens left-to-right in a single pass. When filling cell (row 8, column 3) of a Hard puzzle, it must simultaneously satisfy: its row constraint (7 already-placed digits in the same row), its column constraint (7 digits above it in the same column), its box constraint (several digits in the same 3×3 box), and all the implicit propagation effects these constraints have on the remaining empty cells. A model cannot 'go back' and fix a contradiction discovered three cells later.

Human Sudoku solvers use constraint propagation and backtracking — pencilling in candidates, eliminating possibilities, and undoing choices when they lead to contradictions. To replicate this, a language model would need to be trained on step-by-step reasoning traces ('Cell R3C5 must be 7 because 7 already appears in row 3 at column 2 and in column 5 at row 1, so by elimination...') rather than just (puzzle, solution) pairs. This is called chain-of-thought training, and it fundamentally changes what the model learns to do.

The broader principle: when a scaling approach stops working, stop scaling and reconsider the problem formulation. The task framing — not the model size or data volume — may be what needs to change.

Glossary

- **ANE (Apple Neural Engine):** A dedicated ML accelerator on Apple Silicon chips, co-located with CPU and GPU.
- **Batch size:** Number of training examples processed together before updating model weights.
- **Checkpoint:** Saved snapshot of model weights at a specific training iteration.
- **ChatML:** Conversation format using <|im_start|>role and <|im_end|> tokens (Qwen models).
- **Completion:** The answer portion of a training example.
- **Cosine decay:** LR schedule that reduces learning rate following a cosine curve from high to near-zero.
- **Fine-tuning:** Training a pre-trained model on a new dataset to teach it new behaviour.
- **Gradient descent:** The training algorithm calculates the direction in which to adjust each weight to reduce loss, then takes a small step in that direction.
- **Inference:** Using a trained model to generate output — as opposed to training it. Weights are frozen during inference.
- **Fusing:** Merging a LoRA adapter into base model weights to produce a standalone model.
- **Hyperparameter:** Configuration value set before training: LR, batch size, rank, etc.
- **Learning rate (LR):** Scalar controlling weight update step size. Too high = instability; too low = slow.
- **LoRA (Low-Rank Adaptation):** Fine-tuning technique adding small adapter matrices while freezing base weights.
- **Loss:** Number measuring prediction error. Lower = better.
- **Mask prompt:** Training option that calculates loss only on completion, not the prompt.
- **mlx_lm:** Apple's MLX-based library for training and serving language models on Apple Silicon.
- **Open weights:** A model whose trained parameters are publicly released for download and modification.
- **Overfitting:** When a model memorises training data but fails to generalise (low train loss, high val loss).
- **Parameters:** The numerical weights inside a neural network, learned during training.
- **Prompt:** The input/question portion of a training example.
- **Token:** The basic text unit a language model reads and writes. Usually a word or word-fragment. 'Sudoku' = 1 token.
- **Unified memory:** Apple Silicon's shared RAM pool accessed by CPU, GPU, and ANE on the same chip.
- **Validation loss:** Loss on held-out examples not seen during training — the true measure of generalisation.
- **vllm-mlx:** Apple Silicon port of vLLM: OpenAI-compatible HTTP server for local model inference.
- **Warmup (LR warmup):** Phase at the start of training in which LR ramps up from near-zero to the target value, preventing destabilisation.

Appendix A — Source Code

A.1 finetune/lora_config.yaml

```
model: Qwen/Qwen3-4B
train: true
data: finetune/data
fine_tune_type: lora
lora_parameters:
  rank: 32
  scale: 32.0
  dropout: 0.0
num_layers: -1
batch_size: 8
iters: 3000
learning_rate: 5e-5
lr_schedule:
  name: cosine_decay
  warmup: 100
  warmup_init: 1e-6
  arguments: [5e-5, 2900, 1e-6]
steps_per_report: 100
steps_per_eval: 500
val_batches: 50
max_seq_length: 1024
adapter_path: finetune/adapters-4b
save_every: 500
mask_prompt: true
grad_checkpoint: false
```

A.2 finetune/generate_sudoku.py

```
#!/usr/bin/env python3
import json
import random
from pathlib import Path

# --- Solver (backtracking) ---

def solve(grid: list[list[int]]) -> bool:
    """Solve grid in-place via backtracking. Returns True if solved."""
    for r in range(9):
        for c in range(9):
            if grid[r][c] == 0:
                candidates = _candidates(grid, r, c)
                for v in candidates:
                    grid[r][c] = v
                    if solve(grid):
                        return True
                    grid[r][c] = 0
                return False
    return True

def _candidates(grid, r, c):
    used = set()
    used.update(grid[r])
    used.update(grid[i][c] for i in range(9))
    br, bc = (r // 3) * 3, (c // 3) * 3
    used.update(grid[br+i][bc+j] for i in range(3) for j in range(3))
    return [v for v in range(1, 10) if v not in used]
```

```

def count_solutions(grid: list[list[int]], limit: int = 2) -> int:
    """Count solutions up to limit (stops early at limit)."""
    count = [0]
    def _search(g):
        if count[0] >= limit:
            return
        for r in range(9):
            for c in range(9):
                if g[r][c] == 0:
                    for v in _candidates(g, r, c):
                        g[r][c] = v
                        _search(g)
                        g[r][c] = 0
                    return
        count[0] += 1
    _search([row[:] for row in grid])
    return count[0]

def make_full_grid(rng: random.Random) -> list[list[int]]:
    """Generate a random completed Sudoku grid."""
    grid = [[0]*9 for _ in range(9)]
    # Fill diagonal 3x3 boxes independently (no constraints between them)
    for box in range(3):
        nums = list(range(1, 10))
        rng.shuffle(nums)
        for i in range(3):
            for j in range(3):
                grid[box*3+i][box*3+j] = nums[i*3+j]
    solve(grid)
    return grid

def make_puzzle(grid: list[list[int]], n_remove: int, rng: random.Random) -> list[list[int]]:
    """Remove up to n_remove cells ensuring unique solution."""
    puzzle = [row[:] for row in grid]
    cells = list(range(81))
    rng.shuffle(cells)
    removed = 0
    for idx in cells:
        if removed >= n_remove:
            break
        r, c = idx // 9, idx % 9
        val = puzzle[r][c]
        puzzle[r][c] = 0
        if count_solutions(puzzle) != 1:
            puzzle[r][c] = val # restore
        else:
            removed += 1
    return puzzle

def pick_difficulty(rng: random.Random) -> int:
    """Return number of cells to remove based on difficulty mix."""
    roll = rng.random()
    if roll < 0.25:
        return rng.randint(25, 30) # easy
    elif roll < 0.75:
        return rng.randint(31, 45) # medium
    else:
        return rng.randint(46, 55) # hard

def grid_to_str(grid: list[list[int]], empty: int = 0) -> str:
    """Format grid as 9 space-separated rows."""

```

```

    rows = []
    for row in grid:
        rows.append(" ".join("." if v == empty else str(v) for v in row))
    return "\n".join(rows)

def format_example(puzzle: list[list[int]], solution: list[list[int]]) ->
tuple[str, str]:
    prompt = (
        "<|im_start|>user\n"
        "Solve this Sudoku puzzle. Empty cells are represented by dots.\n\n"
        + grid_to_str(puzzle) + "\n"
        "<|im_end|>\n"
        "<|im_start|>assistant\n"
    )
    completion = grid_to_str(solution) + "\n<|im_end|>"
    return prompt, completion

def generate_examples(n: int, seed: int) -> list[str]:
    rng = random.Random(seed)
    examples = []
    while len(examples) < n:
        solution = make_full_grid(rng)
        n_remove = pick_difficulty(rng)
        puzzle = make_puzzle(solution, n_remove, rng)
        prompt, completion = format_example(puzzle, solution)
        examples.append(json.dumps({"prompt": prompt, "completion":
completion}))
        if len(examples) % 1000 == 0:
            print(f" {len(examples)}/{n}...")
    return examples

def main():
    out_dir = Path(__file__).parent / "data"
    out_dir.mkdir(exist_ok=True)

    print("Generating 20000 train examples (seed=42)...")
    train = generate_examples(20000, seed=42)
    (out_dir / "train.jsonl").write_text("\n".join(train) + "\n")
    print(f"  Written: {out_dir / 'train.jsonl'}")

    print("Generating 2000 valid examples (seed=99)...")
    valid = generate_examples(2000, seed=99)
    (out_dir / "valid.jsonl").write_text("\n".join(valid) + "\n")
    print(f"  Written: {out_dir / 'valid.jsonl'}")

    print("Done.")

if __name__ == "__main__":
    main()

```

A.3 finetune/eval_sudoku.py

```

#!/usr/bin/env python3
"""
Evaluate the fine-tuned Sudoku model against 20 held-out puzzles.
Compares fine-tuned model (via vllm-mlx server) vs base model (via
mlx_lm.generate).

Usage:
# Server must be running:
# vllm-mlx serve finetune/qwen3-1.7b-sudoku-fused --port 7001

```

```
python finetune/eval_sudoku.py
"""

import random
import re
import subprocess
import sys
import json
import requests
from pathlib import Path

sys.path.insert(0, str(Path(__file__).parent))
from generate_sudoku import make_full_grid, make_puzzle, pick_difficulty,
grid_to_str

SERVER_URL = "http://localhost:7001/v1/completions"
MODEL_ID = "finetune/qwen3-4b-sudoku-fused"
BASE_MODEL = "Qwen/Qwen3-4B"
EVAL_SEED = 777
N_PUZZLES = 100

# --- Puzzle generation ---
def generate_eval_puzzles(n: int, seed: int) -> list[tuple]:
    """Generate n (puzzle, solution) pairs using the given random seed."""
    rng = random.Random(seed)
    puzzles = []
    while len(puzzles) < n:
        solution = make_full_grid(rng)
        n_remove = pick_difficulty(rng)
        puzzle = make_puzzle(solution, n_remove, rng)
        puzzles.append((puzzle, solution))
    return puzzles

# --- Prompt formatting ---
def format_prompt(puzzle):
    """Format a puzzle grid as a ChatML prompt for the fine-tuned model."""
    return (
        "<|im_start|>user\n"
        "Solve this Sudoku puzzle. Empty cells are represented by dots.\n\n"
        + grid_to_str(puzzle) + "\n"
        "<|im_end|>\n"
        "<|im_start|>assistant\n"
    )

# --- Response parsing ---
def parse_grid(text: str) -> list[list[int]] | None:
    """Extract the first 9x9 digit grid from model output; returns None if not
    found."""
    lines = [l.strip() for l in text.strip().splitlines() if l.strip()]
    rows = []
    for line in lines:
        nums = re.findall(r'[1-9]', line)
        if len(nums) == 9:
            rows.append([int(n) for n in nums])
        if len(rows) == 9:
            break
    return rows if len(rows) == 9 else None

# --- Validation ---
```

```
def is_valid_solution(puzzle, solution, predicted) -> tuple[bool, str]:
    """Check rows, columns, boxes and given-cell constraints; return (ok,
    reason)."""
    if predicted is None:
        return False, "could not parse grid"
    valid_set = set(range(1, 10))
    for r in range(9):
        if set(predicted[r]) != valid_set:
            return False, f"row {r+1} invalid"
        if {predicted[i][r] for i in range(9)} != valid_set:
            return False, f"col {r+1} invalid"
    for br in range(3):
        for bc in range(3):
            box = {predicted[br*3+i][bc*3+j] for i in range(3) for j in
range(3)}
            if box != valid_set:
                return False, f"box ({br+1},{bc+1}) invalid"
    for r in range(9):
        for c in range(9):
            if puzzle[r][c] != 0 and predicted[r][c] != puzzle[r][c]:
                return False, f"given cell ({r+1},{c+1}) changed"
    return True, "correct"

# — Fine-tuned model inference —————

def query_finetuned(puzzle) -> str | None:
    """Query the fine-tuned model via the vllm-mlx server; returns raw text or
    None on error."""
    try:
        resp = requests.post(SERVER_URL, json={
            "model": MODEL_ID,
            "prompt": format_prompt(puzzle),
            "max_tokens": 512,
            "temperature": 0.0,
            "stop": ["<|im_end|>", "<|im_start|>"],
        }, timeout=60)
        resp.raise_for_status()
        return resp.json()["choices"][0]["text"]
    except Exception as e:
        return None

# — Base model inference —————

def query_base(puzzle) -> str | None:
    """Query the base model via mlx_lm.generate subprocess; returns completion
    text or None."""
    prompt = format_prompt(puzzle)
    try:
        result = subprocess.run(
            [sys.executable, "-m", "mlx_lm", "generate",
             "--model", BASE_MODEL,
             "--prompt", prompt,
             "--max-tokens", "512",
             "--temp", "0.0"],
            capture_output=True, text=True, timeout=120
        )
        output = result.stdout
        # mlx_lm generate prints prompt then completion - strip the prompt
        if prompt in output:
            output = output.split(prompt, 1)[1]
        # strip trailing separator lines (====)
        output = re.sub(r'=.+', '', output, flags=re.DOTALL).strip()
        return output
```

```

    except Exception as e:
        return None

# — Main eval —
def evaluate(puzzles, query_fn, label):
    """Run query_fn over all puzzles, print per-puzzle results, return
    (correct_count, results_list)."""
    correct = 0
    results = []
    print(f"\n{'='*60}")
    print(f"Evaluating: {label}")
    print(f"{'='*60}")
    for i, (puzzle, solution) in enumerate(puzzles):
        raw = query_fn(puzzle)
        predicted = parse_grid(raw) if raw else None
        ok, reason = is_valid_solution(puzzle, solution, predicted)
        status = "✓" if ok else "X"
        empty_cells = sum(1 for r in range(9) for c in range(9) if puzzle[r][c]
== 0)
        print(f"  Puzzle {i+1:3d}: {status} {reason} (empty={empty_cells})")
        if ok:
            correct += 1
            results.append({
                "puzzle_id": i + 1,
                "correct": ok,
                "reason": reason,
                "empty_cells": empty_cells,
                "raw_output": raw[:300] if raw else None,
            })
        print(f"\nResult: {correct}/{len(puzzles)} correct
({100*correct//len(puzzles)}%)")
    return correct, results

def main():
    """Generate held-out puzzles, evaluate the fine-tuned model, and save
    results to eval_results.json."""
    import time
    print(f"Generating {N_PUZZLES} held-out eval puzzles
(seed={EVAL_SEED})...")
    puzzles = generate_eval_puzzles(N_PUZZLES, EVAL_SEED)

    t0 = time.time()
    ft_correct, ft_results = evaluate(puzzles, query_finetuned, f"Fine-tuned
({MODEL_ID})")
    elapsed = time.time() - t0

    # Save results
    out = {
        "model": MODEL_ID,
        "n_puzzles": N_PUZZLES,
        "seed": EVAL_SEED,
        "correct": ft_correct,
        "accuracy": ft_correct / N_PUZZLES,
        "elapsed_seconds": round(elapsed, 1),
        "results": ft_results,
    }
    out_path = Path(__file__).parent / "eval_results.json"
    out_path.write_text(json.dumps(out, indent=2))
    print(f"\nResults saved to {out_path}")
    print(f"Accuracy: {ft_correct}/{N_PUZZLES} ({100*ft_correct//N_PUZZLES}%)
| {elapsed:.0f}s total")

```

```
if __name__ == "__main__":
    main()
```

A.4 sudoku_game.py

```
#!/usr/bin/env python3
"""
Sudoku game with AI solver via vllm-mlx.
Run: python sudoku_game.py
Requires: pip install pygame requests
Server: vllm-mlx serve finetune/qwen3-1.7b-sudoku-fused --port 8000
"""

import random
import time
import threading
import re
import requests
import pygame

# --- Layout constants ---
CELL = 60
GRID = CELL * 9
MARGIN = 20
PANEL_W = 220
WIDTH = GRID + PANEL_W + MARGIN * 3
HEIGHT = GRID + MARGIN * 2
FPS = 30

# --- Colours ---
BG = (245, 245, 240)
GRID_LIGHT = (200, 200, 195)
GRID_BOLD = (60, 60, 60)
GIVEN_FG = (30, 30, 30)
USER_FG = (40, 80, 160)
AI_FG = (20, 140, 60)
ERROR_FG = (200, 40, 40)
SEL_BG = (180, 210, 255)
SAME_BG = (220, 235, 255)
BTN_NORMAL = (70, 130, 180)
BTN_HOVER = (90, 155, 210)
BTN_ACTIVE = (40, 100, 150)
BTN_EASY = (80, 170, 80)
BTN_MED = (200, 150, 40)
BTN_HARD = (190, 60, 60)
WHITE = (255, 255, 255)
BLACK = (0, 0, 0)
TIMER_FG = (80, 80, 80)
STATUS_OK = (30, 140, 30)
STATUS_ERR = (180, 30, 30)
STATUS_INFO = (80, 80, 80)

# --- Difficulty ---
DIFFICULTY = {
    "Easy": (25, 30),
    "Medium": (31, 45),
    "Hard": (46, 55),
}

SERVER_URL = "http://localhost:7001/v1/completions"
MODEL_ID = "finetune/qwen3-1.7b-sudoku-fused"
```

```

# — Sudoku logic —

def _candidates(grid, r, c):
    """Return list of valid digits for cell (r, c) given current grid state."""
    used = set(grid[r])
    used.update(grid[i][c] for i in range(9))
    br, bc = (r // 3) * 3, (c // 3) * 3
    used.update(grid[br+i][bc+j] for i in range(3) for j in range(3))
    return [v for v in range(1, 10) if v not in used]

def solve(grid):
    """Solve grid in-place via backtracking. Returns True if a solution was found."""
    for r in range(9):
        for c in range(9):
            if grid[r][c] == 0:
                for v in _candidates(grid, r, c):
                    grid[r][c] = v
                    if solve(grid):
                        return True
                    grid[r][c] = 0
                return False
    return True

def count_solutions(grid, limit=2):
    """Count distinct solutions up to limit; stops early once limit is reached."""
    count = [0]
    def _search(g):
        if count[0] >= limit:
            return
        for r in range(9):
            for c in range(9):
                if g[r][c] == 0:
                    for v in _candidates(g, r, c):
                        g[r][c] = v
                        _search(g)
                        g[r][c] = 0
                    return
        count[0] += 1
    _search([row[:] for row in grid])
    return count[0]

def make_full_grid(rng):
    """Generate a random fully-solved Sudoku grid using the given RNG."""
    grid = [[0]*9 for _ in range(9)]
    for box in range(3):
        nums = list(range(1, 10))
        rng.shuffle(nums)
        for i in range(3):
            for j in range(3):
                grid[box*3+i][box*3+j] = nums[i*3+j]
    solve(grid)
    return grid

def make_puzzle(solution, n_remove, rng):
    """Remove up to n_remove cells from solution while preserving a unique solution."""
    puzzle = [row[:] for row in solution]
    cells = list(range(81))
    rng.shuffle(cells)
    removed = 0

```

```

    for idx in cells:
        if removed >= n_remove:
            break
        r, c = idx // 9, idx % 9
        val = puzzle[r][c]
        puzzle[r][c] = 0
        if count_solutions(puzzle) != 1:
            puzzle[r][c] = val
        else:
            removed += 1
    return puzzle

def new_puzzle(difficulty="Medium"):
    """Generate a fresh (puzzle, solution) pair at the requested difficulty level."""
    rng = random.Random()
    solution = make_full_grid(rng)
    lo, hi = DIFFICULTY[difficulty]
    n_remove = rng.randint(lo, hi)
    puzzle = make_puzzle(solution, n_remove, rng)
    return puzzle, solution

# — AI solver —
def grid_to_prompt(puzzle):
    """Format puzzle grid as a ChatML prompt for the vllm-mlx inference server."""
    rows = []
    for row in puzzle:
        rows.append(" ".join("." if v == 0 else str(v) for v in row))
    grid_str = "\n".join(rows)
    return (
        "<|im_start|>user\n"
        "Solve this Sudoku puzzle. Empty cells are represented by dots.\n\n"
        + grid_str + "\n"
        "<|im_end|>\n"
        "<|im_start|>assistant\n"
    )

def parse_grid_response(text):
    """Extract 9x9 grid from model response. Returns list[list[int]] or None."""
    lines = [l.strip() for l in text.strip().splitlines() if l.strip()]
    rows = []
    for line in lines:
        nums = re.findall(r'[1-9]', line)
        if len(nums) == 9:
            rows.append([int(n) for n in nums])
        if len(rows) == 9:
            break
    return rows if len(rows) == 9 else None

def call_ai(puzzle, on_result, on_error):
    """Call vllm-mlx server in background thread."""
    def _run():
        try:
            prompt = grid_to_prompt(puzzle)
            resp = requests.post(SERVER_URL, json={
                "model": MODEL_ID,
                "prompt": prompt,
                "max_tokens": 512,
                "temperature": 0.0,
                "stop": ["<|im_end|>", "<|im_start|>"],

```

```

        }, timeout=60)
        resp.raise_for_status()
        text = resp.json()["choices"][0]["text"]
        grid = parse_grid_response(text)
        if grid:
            on_result(grid)
        else:
            on_error("AI response couldn't be parsed")
    except requests.exceptions.ConnectionError:
        on_error("Server not running - start vllm-mlx first")
    except Exception as e:
        on_error(str(e)[:60])
    threading.Thread(target=_run, daemon=True).start()

# --- UI helpers ---

def draw_button(surf, rect, label, color, hover=False, font=None):
    """Draw a rounded rectangle button with centred label onto surf."""
    c = BTN_HOVER if hover else color
    pygame.draw.rect(surf, c, rect, border_radius=6)
    pygame.draw.rect(surf, (0, 0, 0, 60), rect, width=1, border_radius=6)
    txt = font.render(label, True, WHITE)
    surf.blit(txt, txt.get_rect(center=rect.center))

def is_valid_placement(grid, r, c, v):
    """Return True if placing digit v at (r, c) violates no row/column/box
    constraint."""
    if v in grid[r]:
        return False
    if v in [grid[i][c] for i in range(9)]:
        return False
    br, bc = (r // 3) * 3, (c // 3) * 3
    for i in range(3):
        for j in range(3):
            if grid[br+i][bc+j] == v:
                return False
    return True

# --- Main game ---

def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Sudoku - AI Solver")
    clock = pygame.time.Clock()

    font_lg = pygame.font.SysFont("helveticaneue", 32, bold=True)
    font_md = pygame.font.SysFont("helveticaneue", 22)
    font_sm = pygame.font.SysFont("helveticaneue", 16)
    font_btn = pygame.font.SysFont("helveticaneue", 18, bold=True)

    # Game state
    difficulty = "Medium"
    puzzle, solution = new_puzzle(difficulty)
    given = [[puzzle[r][c] != 0 for c in range(9)] for r in range(9)]
    user_grid = [row[:] for row in puzzle]
    ai_grid = None
    selected = None

    # Timers
    puzzle_start = time.time()
    user_time = None # set when user completes or AI solves
    ai_time = None

```

```

# UI state
status_msg = ""
status_color = STATUS_INFO
ai_thinking = False

# Grid origin
ox = MARGIN
oy = MARGIN

def reset(diff=None):
    nonlocal difficulty, puzzle, solution, given, user_grid, ai_grid
    nonlocal selected, puzzle_start, user_time, ai_time
    nonlocal status_msg, status_color, ai_thinking
    if diff:
        difficulty = diff
    puzzle, solution = new_puzzle(difficulty)
    given = [[puzzle[r][c] != 0 for c in range(9)] for r in range(9)]
    user_grid = [row[:] for row in puzzle]
    ai_grid = None
    selected = None
    puzzle_start = time.time()
    user_time = None
    ai_time = None
    status_msg = ""
    status_color = STATUS_INFO
    ai_thinking = False

def check_complete():
    nonlocal user_time, status_msg, status_color
    for r in range(9):
        for c in range(9):
            if user_grid[r][c] == 0:
                return
            if user_grid[r][c] != solution[r][c]:
                return
    user_time = time.time() - puzzle_start
    status_msg = f"Solved in {user_time:.1f}s!"
    status_color = STATUS_OK

def on_ai_result(grid):
    nonlocal ai_grid, ai_time, status_msg, status_color, ai_thinking
    ai_grid = grid
    ai_time = time.time() - ai_start[0]
    ai_thinking = False
    correct = sum(
        1 for r in range(9) for c in range(9)
        if grid[r][c] == solution[r][c]
    )
    status_msg = f"AI: {correct}/81 correct ({ai_time:.1f}s)"
    status_color = STATUS_OK if correct == 81 else STATUS_ERR

def on_ai_error(msg):
    nonlocal ai_thinking, status_msg, status_color
    ai_thinking = False
    status_msg = msg
    status_color = STATUS_ERR

ai_start = [0.0]

running = True
while running:
    mx, my = pygame.mouse.get_pos()

```

```

# — Panel button rects —
px = GRID + MARGIN * 2
btn_easy = pygame.Rect(px, oy + 10, PANEL_W - 10, 38)
btn_med = pygame.Rect(px, oy + 58, PANEL_W - 10, 38)
btn_hard = pygame.Rect(px, oy + 106, PANEL_W - 10, 38)
btn_new = pygame.Rect(px, oy + 170, PANEL_W - 10, 42)
btn_solve = pygame.Rect(px, oy + 224, PANEL_W - 10, 42)
btn_clear = pygame.Rect(px, oy + 278, PANEL_W - 10, 36)

# — Events —
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        running = False

    elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        # Grid click
        gx = (mx - ox) // CELL
        gy = (my - oy) // CELL
        if 0 <= gx < 9 and 0 <= gy < 9:
            selected = (gy, gx)
        # Buttons
        elif btn_easy.collidepoint(mx, my):
            reset("Easy")
        elif btn_med.collidepoint(mx, my):
            reset("Medium")
        elif btn_hard.collidepoint(mx, my):
            reset("Hard")
        elif btn_new.collidepoint(mx, my):
            reset()
        elif btn_clear.collidepoint(mx, my):
            # Clear user entries, keep givens
            for r in range(9):
                for c in range(9):
                    if not given[r][c]:
                        user_grid[r][c] = 0
            ai_grid = None
            status_msg = ""
        elif btn_solve.collidepoint(mx, my) and not ai_thinking:
            ai_thinking = True
            ai_start[0] = time.time()
            status_msg = "Asking AI..."
            status_color = STATUS_INFO
            call_ai(user_grid, on_ai_result, on_ai_error)

    elif event.type == pygame.KEYDOWN and selected:
        r, c = selected
        if not given[r][c] and ai_grid is None:
            if event.key in (pygame.K_BACKSPACE, pygame.K_DELETE,
pygame.K_0):
                user_grid[r][c] = 0
            elif pygame.K_1 <= event.key <= pygame.K_9:
                v = event.key - pygame.K_0
                user_grid[r][c] = v
                check_complete()
        # Arrow navigation
        dr = {pygame.K_UP: -1, pygame.K_DOWN: 1}.get(event.key, 0)
        dc = {pygame.K_LEFT: -1, pygame.K_RIGHT: 1}.get(event.key, 0)
        if dr or dc:
            selected = (max(0, min(8, r+dr)), max(0, min(8, c+dc)))

# — Draw background —
screen.fill(BG)

```

```

# — Draw grid cells —
active_grid = ai_grid if ai_grid else user_grid
sel_val = active_grid[selected[0]][selected[1]] if selected else 0

for r in range(9):
    for c in range(9):
        cx = ox + c * CELL
        cy = oy + r * CELL
        rect = pygame.Rect(cx, cy, CELL, CELL)

        # Cell background
        if selected and (r, c) == selected:
            pygame.draw.rect(screen, SEL_BG, rect)
        elif sel_val and active_grid[r][c] == sel_val and not ai_grid:
            pygame.draw.rect(screen, SAME_BG, rect)
        elif selected and (selected[0] == r or selected[1] == c or
                           (r//3 == selected[0]//3 and c//3 ==
selected[1]//3)):
            pygame.draw.rect(screen, (235, 235, 228), rect)

        # Cell value
        val = active_grid[r][c]
        if val:
            if ai_grid:
                color = AI_FG if val == solution[r][c] else ERROR_FG
            elif given[r][c]:
                color = GIVEN_FG
            else:
                color = USER_FG if is_valid_placement(
                    [row[:] for row in user_grid], r, c, val
                ) else ERROR_FG
            txt = font_lg.render(str(val), True, color)
            screen.blit(txt, txt.get_rect(center=(cx + CELL//2, cy +
CELL//2)))

# — Draw grid lines —
for i in range(10):
    bold = i % 3 == 0
    lw = 3 if bold else 1
    color = GRID_BOLD if bold else GRID_LIGHT
    pygame.draw.line(screen, color, (ox + i*CELL, oy), (ox + i*CELL, oy
+ GRID), lw)
    pygame.draw.line(screen, color, (ox, oy + i*CELL), (ox + GRID, oy +
i*CELL), lw)

# — Panel —
# Difficulty buttons
for btn, label, color in [
    (btn_easy, "Easy", BTN_EASY),
    (btn_med, "Medium", BTN_MED),
    (btn_hard, "Hard", BTN_HARD),
]:
    selected_diff = label == difficulty
    c = BTN_ACTIVE if selected_diff else color
    draw_button(screen, btn, label, c, btn.collidepoint(mx, my),
font_btn)

    draw_button(screen, btn_new, "New Puzzle", BTN_NORMAL,
btn_new.collidepoint(mx, my), font_btn)
    draw_button(screen, btn_solve, "Solve with AI" if not ai_thinking else
"Thinking...",

```

```

        BTN_ACTIVE if ai_thinking else BTN_NORMAL,
btn_solve.collidepoint(mx, my), font_btn)
    draw_button(screen, btn_clear, "Clear",          (130, 130, 130),
btn_clear.collidepoint(mx, my), font_btn)

    # — Timers —
    ty = btn_clear.bottom + 20
    elapsed = user_time if user_time else time.time() - puzzle_start
    timer_label = font_sm.render("Your time", True, TIMER_FG)
    timer_val = font_md.render(f"{elapsed:.1f}s", True, TIMER_FG)
    screen.blit(timer_label, (px, ty))
    screen.blit(timer_val, (px, ty + 18))

    if ai_time:
        ty2 = ty + 50
        ai_label = font_sm.render("AI time", True, TIMER_FG)
        ai_val = font_md.render(f"{ai_time:.1f}s", True, AI_FG)
        screen.blit(ai_label, (px, ty2))
        screen.blit(ai_val, (px, ty2 + 18))

    # — Status message —
    if status_msg:
        sy = HEIGHT - MARGIN - 20
        lines = [status_msg[i:i+26] for i in range(0, len(status_msg), 26)]
        for i, line in enumerate(lines[:3]):
            stxt = font_sm.render(line, True, status_color)
            screen.blit(stxt, (px, sy - (len(lines)-1-i)*18))

    pygame.display.flip()
    clock.tick(FPS)

    pygame.quit()

if __name__ == "__main__":
    main()

```

Appendix B — Commands Reference

B.0 Reproducibility Metadata

For full reproducibility, record these details from your own environment before running. The versions below reflect the environment used in this experiment.

```
# Check your exact package versions after installation:
pip show mlx mlx-lm vllm-mlx pygame requests python-docx | grep -E
"Name|Version"

# Environment used in this experiment (approximate):
# mlx                                0.24.x
# mlx-lm                             0.22.x
# vllm-mlx                           0.x (May 2026)
# pygame                             2.6.x
# requests                           2.32.x
# Python                             3.12.x (arm64)
# macOS                               26 Darwin 25.4.0
# Hardware                            Apple M5 Max, 128 GB unified memory

# To capture your exact pip state:
pip freeze > requirements_frozen.txt
```

Additional details relevant to reproducibility: the tokenizer used is the standard Qwen3 tokenizer bundled with the model weights (downloaded from HuggingFace). No custom tokenizer modifications were made. The base model was not quantized before fine-tuning — full bf16 weights were used during training. The fused model (finetune/qwen3-4b-sudoku-fused) is also bf16.

B.1 Environment Setup

```
$(brew --prefix python@3.12)/bin/python3.12 -m venv ~/Develop/vllm-mlx
source ~/Develop/vllm-mlx/bin/activate
pip install vllm-mlx mlx-lm pygame requests
```

B.2 Dataset Generation

```
python finetune/generate_sudoku.py
# Writes finetune/data/train.jsonl (20k) and valid.jsonl (2k)
```

B.3 Training

```
python -m mlx_lm.lora --config finetune/lora_config.yaml
# Checkpoints saved to finetune/adapters-4b/ every 500 iters
# Key output: Iter N: Train loss X.XXX, Val loss X.XXX, Tokens-per-sec: YYY
```

B.4 Fusing Adapter into Base Model

```
# 4B model
python -m mlx_lm.fuse \
  --model Qwen/Qwen3-4B \
  --adapter-path finetune/adapters-4b \
  --save-path finetune/qwen3-4b-sudoku-fused

# 1.7B model
python -m mlx_lm.fuse \
  --model Qwen/Qwen3-1.7B \
  --adapter-path finetune/adapters \
  --save-path finetune/qwen3-1.7b-sudoku-fused
```

B.5 Serving the Fine-Tuned Model

```
# Serve on port 7001 (avoids conflict with default port 8000)
vllm-mlx serve finetune/qwen3-4b-sudoku-fused --port 7001

# Health check
curl http://localhost:7001/health
```

B.6 Evaluation (100 puzzles)

```
python finetune/eval_sudoku.py
# Results saved to finetune/eval_results.json
```

B.7 Starting the Game

```
python sudoku_game.py
```

Summary and What to Do Next

Seven training runs. Two days. One laptop. A model that went from 0% to 81% Easy Sudoku accuracy — and hit a hard wall at 0% on Hard puzzles that no amount of additional data or model scaling could move. That wall is not a failure. It is the most important result in the entire experiment.

The journey proved four things:

- A consumer MacBook with an M-series chip and enough unified memory is a fully capable fine-tuning machine—no cloud GPU account required.
- The pipeline is reproducible in under two hours once the mistakes are known. The lessons in Section 11 are the shortcut.
- Structured generation tasks (where every output token matters) need val loss below 0.03 and per-token accuracy near 99%. Standard language modelling loss thresholds do not apply.
- Fine-tuning adjusts what patterns a model has learned. It does not change the model's computational structure. When the task requires search and backtracking, the architecture itself — not the training data — is the bottleneck.

The Unfinished Problem: Your Challenge

Hard Sudoku accuracy is 0%. It was 0% for both the 1.7B and 4B models. It did not move with 4× more training data. This document has diagnosed the cause and outlined the most promising solution — chain-of-thought training — but has not implemented it. That is deliberate.

The open problem is this: can a language model learn to solve Hard Sudoku if it is trained on reasoning traces rather than direct (puzzle, solution) pairs? A chain-of-thought dataset would contain examples like:

```
PROMPT:
<|im_start|>user
Solve this Sudoku puzzle. Show your reasoning step by step.

5 3 . . 7 . . . .
...
<|im_end|>
<|im_start|>assistant

COMPLETION (reasoning trace + solution):
Step 1: Row 1 has 5, 3, 7. Missing: 1, 2, 4, 6, 8, 9.
Step 2: Column 1 has 5, 6, 8, 4, 7. Cell (R1,C3) cannot be 5, 6, 8, 4, 7.
Step 3: Box (1,1) has 5, 3, 6, 9, 8. Cell (R1,C3) cannot be 5, 3, 6, 9, 8.
Step 4: By elimination, cell (R1,C3) must be 4.
...
[full 9x9 solution grid]
<|im_end|>
```

The key question is whether externalising the constraint-propagation reasoning as training text gives the model enough System 2 scaffolding to solve puzzles it could not solve with direct pattern matching. This experiment has everything needed to run the tests: the dataset generator, the training pipeline, and the evaluation script. Only the reasoning trace generator is missing.

Concept — Mini-Lab Assignment: Build the Chain-of-Thought Dataset: Your task: write a `generate_sudoku_cot.py` script that produces training examples in the format above. The script must: (1) generate a puzzle using the existing `make_puzzle()` function from `generate_sudoku.py`; (2) solve it using a constraint-propagation solver that records each deduction step as a human-readable sentence; (3) format the puzzle, reasoning trace, and solution as a prompt/completion pair. Train on 5,000 examples of this format using the same `lora_config.yaml` pipeline (increase `max_seq_length` to 2048; reduce `batch_size` to 4). Evaluate with the same `eval_sudoku.py` script. Hypothesis to test: Does Hard accuracy move above 0%? Bonus: Does it improve or hurt Easy accuracy? The entire experiment should be runnable in under two hours on the same hardware. A complete sample solution is provided in Appendix C.

What Should You Fine-Tune?

The best first fine-tuning project has four properties: it has a clear correct answer (so you can measure success automatically), the input-output pairs are easy to generate programmatically, the output is short (under 500 tokens), and you actually care about the result. Below are five concrete starting points, ordered from easiest to most ambitious.

Project	What you fine-tune on	Why it works well	Suggested model	Evaluation method
SQL from English	Natural language question → SQL query pairs generated from a schema	Exact match evaluation; pairs are easy to generate; output is short	Qwen3-1.7B or 4B	Execute the query, compare the result set to the reference
JSON extraction	Raw text (emails, receipts, articles) → structured JSON with specific fields	JSON validity is machine-checkable; pairs easy to synthesise from templates	Qwen3-1.7B	JSON schema validation + per-field accuracy
Code style normaliser	Inconsistently formatted code → canonical style (your team's conventions)	Before/after pairs come directly from your git history via git diff	Qwen3-4B	Diff distance + formatter check (e.g. ruff/black)
Domain Q&A	Question-answer pairs extracted from your own documentation or notes	Your data, your domain — the base model has no coverage of it	Qwen3-4B or 7B	Citation recall + human spot-check

Chain-of-thought Sudoku (this project continued)	Puzzle → step-by-step constraint reasoning → solution triples	Direct extension of this work; likely fixes Hard puzzle accuracy	Qwen3-4B	Final grid validity + reasoning-step consistency
---	---	--	----------	--

When Not to Fine-Tune

Fine-tuning is not the answer to every AI problem. Before investing training time, check whether a simpler approach solves it first.

Use fine-tuning when...	Prefer RAG / prompting / rules / tools when...
You need consistent output style or format	You need access to fresh or real-time facts
You have many high-quality input-output examples (hundreds to thousands)	You have few examples (< 50) — use few-shot prompting instead
The output format is stable and well-defined	The task changes frequently — retraining is costly
Latency and local control matter (no API calls)	Exact factual correctness is required — retrieval is more reliable
The base model has no coverage of your domain or notation	A rule-based or symbolic system solves it perfectly (like Sudoku backtracking)
You need the model to internalise a repeated behaviour	You need traceable, auditable reasoning steps — retrieval-augmented generation is more reliable

The Sudoku experiment is a perfect example of the last row: a symbolic backtracking solver achieves 100% accuracy in milliseconds. The reason to fine-tune anyway is educational — to understand how fine-tuning behaves when outputs are structured and verifiable. In a production context, you would use the solver.

How This Document Was Started: A Note on Asking Claude

Before writing a single line of training code, I asked Claude (Anthropic’s AI assistant) to help design the experiment. The initial prompt was deliberately open-ended:

“I want to fine-tune a language model on Apple Silicon for the first time. I have an M5 Max with 128 GB. I want to teach it to solve Sudoku puzzles. What model should I start with, what framework, and what does the pipeline look like?”

From that single question, Claude outlined the full pipeline: generate a dataset, use mlx_lm.lora on an appropriately sized Qwen3 model, fuse the adapter, serve with vllm-mlx, and evaluate with a held-out set. The brainstorming session identified the key risks upfront (mask_prompt, LR decay, dataset format) — though running into them in practice was still the best teacher.

Claude was used as a planning assistant, not as an authority. The actual learning came from running the pipeline, observing failures, and debugging the mismatch between expectation and result. AI can scaffold a path; only empirical work teaches you to walk it.

If you want to start your own fine-tuning project, a good opening question follows the same pattern: describe your hardware, your task, your data source, and your success criterion. Ask for the pipeline first, not the code. Once the pipeline makes sense end-to-end, the code follows naturally — and you will know what each piece is for before you write it.

References

The following papers, repositories, and tools were directly used in this experiment.

Papers

LoRA: Low-Rank Adaptation of Large Language Models. Edward Hu et al., 2021. The original paper introducing LoRA. Describes the mathematical basis for inserting low-rank adapter matrices. <https://arxiv.org/abs/2106.09685>

Qwen3 Technical Report. Qwen Team, Alibaba Cloud, 2025. Describes the Qwen3 model family, including the 1.7B and 4B models used in this experiment. <https://arxiv.org/abs/2505.09388>

Software and Tools

MLX — Apple's Machine Learning Framework. The low-level numerical computing framework underlying mlx_lm. Optimised for Apple Silicon unified memory architecture.

<https://github.com/ml-explore/mlx>

mlx-lm — Language Model Training and Inference on MLX. Provides mlx_lm.lora (fine-tuning), mlx_lm.fuse (adapter merging), and mlx_lm.generate (CLI inference). The training framework used in this project. <https://github.com/ml-explore/mlx-lm>

vllm-mlx — vLLM on Apple Silicon. Community-maintained project providing a vLLM-style OpenAI-compatible HTTP server on Apple Silicon via MLX. The inference server used in this project. <https://pypi.org/project/vllm-mlx/>

Qwen3 Model Weights on HuggingFace. The Qwen/Qwen3-1.7B and Qwen/Qwen3-4B model weights were used as starting points. Downloaded automatically by mlx_lm on first training run. <https://huggingface.co/Qwen>

pygame — Python Game Development Library. Used to build the interactive Sudoku game in sudoku_game.py. <https://www.pygame.org>

python-docx — Word Document Generation. Python library used to generate this document programmatically. <https://python-docx.readthedocs.io>

Further Reading

The Illustrated Transformer — Jay Alammar. The most accessible visual explanation of the transformer architecture that underlies all language models discussed here.

<https://jalammar.github.io/illustrated-transformer/>

Andrej Karpathy: Let's build GPT from scratch. A 2-hour video walkthrough on building a small language model from scratch in Python. Best resource for understanding what training actually does mechanistically. <https://youtu.be/kCc8FmEb1nY>

Lilian Weng: Parameter-Efficient Fine-Tuning. Comprehensive blog post covering LoRA, prefix tuning, and other PEFT methods with mathematical detail.
<https://lilianweng.github.io/posts/2023-01-27-the-transformer-family-v2/>

HuggingFace LoRA Conceptual Guide. HuggingFace documentation explaining LoRA and the PEFT library from a practitioner perspective.
https://huggingface.co/docs/peft/conceptual_guides/lora

Appendix C — Mini-Lab Sample Solution: `generate_sudoku_cot.py`

The following is a complete sample solution for the Mini-Lab Assignment from the Summary section. It implements a reasoning-trace generator that uses three logical deduction strategies before falling back to annotated guessing:

- **Naked single:** only one candidate digit remains for a cell after eliminating all row, column, and box conflicts.
- **Hidden single:** a digit has only one valid cell remaining within an entire row, column, or 3×3 box.
- **Naked pair:** two cells in a unit share the same two candidates — those digits can be eliminated from all other cells in that unit.
- **Annotated backtracking:** when logic alone cannot progress, the solver makes an explicit guess, records it as a step, and continues. If the guess leads to a contradiction, it backtracks and records that too.

The key design principle: every deduction is recorded as a human-readable sentence before the digit is placed. This means the completion token sequence the model learns to produce is the reasoning trace first, then the final grid, giving it explicit System 2 scaffolding to work through constraints step by step.

Sample output for a medium puzzle (38 empty cells):

```
Step 1: Naked single: R1C2 = 7 (only candidate remaining in cell)
Step 2: Hidden single: R2C4 = 3 (only cell in row 2 that can hold 3)
Step 3: Naked pair {4,8} at R3C1 and R3C7 — eliminated 4 and 8 from other cells
in row 3
Step 4: Naked single: R3C5 = 2 (only candidate remaining in cell)
...
Step 38: Naked single: R9C9 = 6 (only candidate remaining in cell)

Solution:
5 7 4 1 9 6 3 2 8
2 6 9 3 8 5 1 4 7
...
8 3 1 7 4 2 9 5 6
```

Training procedure: use the same `lora_config.yaml` as the main experiment, but point data: at the CoT dataset. Note that reasoning traces are ~2,000 characters per example vs ~400 for direct (puzzle, solution) pairs, so `max_seq_length` should be increased to 2048 and `batch_size` reduced to 4 to fit in memory.

```
# lora_config_cot.yaml changes from the baseline:
data: finetune/data          # train_cot.jsonl and valid_cot.jsonl must be
renamed                      # or symlinked to train.jsonl and valid.jsonl
max_seq_length: 2048         # reasoning traces are ~5x longer than direct
pairs
batch_size: 4                # reduced from 8 to fit longer sequences
iters: 3000                  # same as baseline
learning_rate: 5e-5          # same as baseline
```

Evaluation: the same `eval_sudoku.py` script works unchanged — it only inspects the final 9×9 grid, which the CoT completion still ends with. The reasoning steps before 'Solution:' are naturally stripped by the parser looking for 9 lines of 9 digits.

C.1 finetune/generate_sudoku_cot.py

```
#!/usr/bin/env python3
"""
Mini-Lab Assignment – Sample Solution
=====
Generates chain-of-thought (CoT) Sudoku training examples.

Each example is a (prompt, completion) pair where the completion contains:
  1. Step-by-step constraint-propagation reasoning
  2. The final 9x9 solution grid

The reasoning traces four deduction strategies in order of complexity:
  - Naked single: only one candidate remains for a cell
  - Hidden single: a digit has only one valid cell in a row/col/box
  - Naked pair: two cells in a unit share the same two candidates
  (elimination)
  - Backtracking: when logic alone cannot progress, a guess is made with
  explicit annotation

Usage:
  python finetune/generate_sudoku_cot.py
  # Writes finetune/data/train_cot.jsonl (5k) and valid_cot.jsonl (500)
"""

import copy
import json
import random
from pathlib import Path

import sys
sys.path.insert(0, str(Path(__file__).parent))
from generate_sudoku import make_full_grid, make_puzzle, pick_difficulty,
grid_to_str

# — Candidate tracking

def all_candidates(grid: list[list[int]]) -> list[list[set]]:
    """Return a 9x9 grid of candidate sets for each empty cell."""
    cands = [[set() for _ in range(9)] for _ in range(9)]
    for r in range(9):
        for c in range(9):
            if grid[r][c] == 0:
                used = set(grid[r])
                used.update(grid[i][c] for i in range(9))
                br, bc = (r // 3) * 3, (c // 3) * 3
                used.update(grid[br+i][bc+j] for i in range(3) for j in
range(3))
                cands[r][c] = set(range(1, 10)) - used
    return cands

def _box_cells(r: int, c: int) -> list[tuple[int, int]]:
    br, bc = (r // 3) * 3, (c // 3) * 3
```

```

    return [(br + i, bc + j) for i in range(3) for j in range(3)]

# — Deduction strategies (return step text + whether progress was made)
# —————

def find_naked_single(
    grid: list[list[int]],
    cands: list[list[set]],
    steps: list[str],
) -> bool:
    """Fill any cell with exactly one candidate. Returns True if any found."""
    for r in range(9):
        for c in range(9):
            if grid[r][c] == 0 and len(cands[r][c]) == 1:
                v = next(iter(cands[r][c]))
                grid[r][c] = v
                cands[r][c] = set()
                # Remove v from peers
                for i in range(9):
                    cands[r][i].discard(v)
                    cands[i][c].discard(v)
                for br2, bc2 in _box_cells(r, c):
                    cands[br2][bc2].discard(v)
                steps.append(
                    f"Naked single: R{r+1}C{c+1} = {v} "
                    f"(only candidate remaining in cell)"
                )
                return True
    return False

def find_hidden_single(
    grid: list[list[int]],
    cands: list[list[set]],
    steps: list[str],
) -> bool:
    """Find a digit that appears in only one cell within a unit. Returns True
    if any found."""

    # Check rows
    for r in range(9):
        for v in range(1, 10):
            positions = [c for c in range(9) if grid[r][c] == 0 and v in
cands[r][c]]
            if len(positions) == 1:
                c = positions[0]
                grid[r][c] = v
                cands[r][c] = set()
                for i in range(9):
                    cands[r][i].discard(v)
                    cands[i][c].discard(v)
                for br2, bc2 in _box_cells(r, c):
                    cands[br2][bc2].discard(v)
                steps.append(
                    f"Hidden single: R{r+1}C{c+1} = {v} "
                    f"(only cell in row {r+1} that can hold {v})"
                )
                return True

    # Check columns
    for c in range(9):
        for v in range(1, 10):

```

```

        positions = [r for r in range(9) if grid[r][c] == 0 and v in
cands[r][c]]
        if len(positions) == 1:
            r = positions[0]
            grid[r][c] = v
            cands[r][c] = set()
            for i in range(9):
                cands[r][i].discard(v)
                cands[i][c].discard(v)
            for br2, bc2 in _box_cells(r, c):
                cands[br2][bc2].discard(v)
            steps.append(
                f"Hidden single: R{r+1}C{c+1} = {v} "
                f"(only cell in column {c+1} that can hold {v})"
            )
            return True

    # Check 3x3 boxes
    for box_r in range(3):
        for box_c in range(3):
            cells = [(box_r*3+i, box_c*3+j) for i in range(3) for j in
range(3)]
            for v in range(1, 10):
                positions = [(r, c) for r, c in cells if grid[r][c] == 0 and v
in cands[r][c]]
                if len(positions) == 1:
                    r, c = positions[0]
                    grid[r][c] = v
                    cands[r][c] = set()
                    for i in range(9):
                        cands[r][i].discard(v)
                        cands[i][c].discard(v)
                    for br2, bc2 in _box_cells(r, c):
                        cands[br2][bc2].discard(v)
                    steps.append(
                        f"Hidden single: R{r+1}C{c+1} = {v} "
                        f"(only cell in box ({box_r+1},{box_c+1}) that can hold
{v})"
                    )
                    return True

            return False

    return False

def find_naked_pair(
    grid: list[list[int]],
    cands: list[list[set]],
    steps: list[str],
) -> bool:
    """Find two cells in a unit with the same two candidates; eliminate from
peers."""

    def _apply_pair(unit_cells, label):
        pairs = [(r, c) for r, c in unit_cells if grid[r][c] == 0 and
len(cands[r][c]) == 2]
        for i in range(len(pairs)):
            for j in range(i + 1, len(pairs)):
                r1, c1 = pairs[i]
                r2, c2 = pairs[j]
                if cands[r1][c1] == cands[r2][c2]:
                    shared = cands[r1][c1]
                    a, b = sorted(shared)
                    eliminated = False

```

```

        for r3, c3 in unit_cells:
            if (r3, c3) in ((r1, c1), (r2, c2)):
                continue
            before = len(cands[r3][c3])
            cands[r3][c3] -= shared
            if len(cands[r3][c3]) < before:
                eliminated = True
        if eliminated:
            steps.append(
                f"Naked pair {{{a},{b}}} at R{r1+1}C{c1+1} and
R{r2+1}C{c2+1} "
                f"-- eliminated {a} and {b} from other cells in
{label}"
            )
            return True
        return False

    for r in range(9):
        if _apply_pair([(r, c) for c in range(9)], f"row {r+1}"):
            return True
    for c in range(9):
        if _apply_pair([(r, c) for r in range(9)], f"column {c+1}"):
            return True
    for box_r in range(3):
        for box_c in range(3):
            cells = [(box_r*3+i, box_c*3+j) for i in range(3) for j in
range(3)]
            if _apply_pair(cells, f"box ({box_r+1},{box_c+1})"):
                return True
    return False

# — Main solver with trace generation —————

def solve_with_trace(
    puzzle: list[list[int]],
    max_steps: int = 120,
    max_guesses: int = 3,
    _guesses_used: int = 0,
) -> tuple[list[list[int]] | None, list[str]]:
    """
    Solve puzzle using logic strategies + limited recursive backtracking.
    Returns (solved_grid, steps) or (None, steps) if unsolvable within limits.

    max_guesses is enforced across recursive calls via _guesses_used.
    Each guess level recurses into solve_with_trace, so multi-level guessing
    (e.g. a puzzle that needs two consecutive guesses) is handled correctly.
    """
    grid = copy.deepcopy(puzzle)
    cands = all_candidates(grid)
    steps: list[str] = []

    # Apply logic strategies until no further progress
    while True:
        if all(grid[r][c] != 0 for r in range(9) for c in range(9)):
            return grid, steps
        if len(steps) >= max_steps:
            return None, steps
        if find_naked_single(grid, cands, steps):
            continue
        if find_hidden_single(grid, cands, steps):
            continue
        if find_naked_pair(grid, cands, steps):

```

```

        continue
    break # no more logical progress – need a guess

# Enforce guess budget before attempting backtracking
if _guesses_used >= max_guesses:
    return None, steps

# Find the cell with the fewest candidates (MRV heuristic)
best_r, best_c = -1, -1
best_len = 10
for r in range(9):
    for c in range(9):
        if grid[r][c] == 0 and 0 < len(cands[r][c]) < best_len:
            best_len = len(cands[r][c])
            best_r, best_c = r, c

if best_r == -1:
    return None, steps # no candidates – contradiction

candidates_snapshot = sorted(cands[best_r][best_c])
for v in candidates_snapshot:
    steps.append(
        f"Guess: R{best_r+1}C{best_c+1} = {v} "
        f"(trying from candidates {candidates_snapshot})"
    )
    grid_copy = copy.deepcopy(grid)
    cands_copy = copy.deepcopy(cands)
    grid_copy[best_r][best_c] = v
    cands_copy[best_r][best_c] = set()
    for i in range(9):
        cands_copy[best_r][i].discard(v)
        cands_copy[i][best_c].discard(v)
    for br2, bc2 in _box_cells(best_r, best_c):
        cands_copy[br2][bc2].discard(v)

    # Reconstruct a partial puzzle from the current grid state for
recursion
    sub_puzzle = copy.deepcopy(grid_copy)
    sub_result, sub_steps = solve_with_trace(
        sub_puzzle, max_steps=max_steps,
        max_guesses=max_guesses, _guesses_used=_guesses_used + 1,
    )
    if sub_result is not None:
        steps.extend(sub_steps)
        return sub_result, steps
    steps.append(f"Backtrack: guess R{best_r+1}C{best_c+1} = {v} led to
contradiction")

    return None, steps

# — Example formatting

```

```

def format_cot_example(
    puzzle: list[list[int]],
    steps: list[str],
    solution: list[list[int]],
) -> tuple[str, str]:
    prompt = (
        "<|im_start|>user\n"
        "Solve this Sudoku puzzle step by step. "

```

```

        "Show your reasoning for each cell before writing the final
solution.\n\n"
        + grid_to_str(puzzle) + "\n"
        "<|im_end|>\n"
        "<|im_start|>assistant\n"
    )
    reasoning = "\n".join(f"Step {i+1}: {s}" for i, s in enumerate(steps))
    completion = (
        reasoning
        + "\n\nSolution:\n"
        + grid_to_str(solution)
        + "\n<|im_end|>"
    )
    return prompt, completion

```

— Dataset generation

```

def generate_cot_examples(
    n: int,
    seed: int,
    max_steps: int = 120,
) -> list[str]:
    rng = random.Random(seed)
    examples = []
    attempts = 0

    while len(examples) < n:
        attempts += 1
        solution = make_full_grid(rng)
        n_remove = pick_difficulty(rng)
        puzzle = make_puzzle(solution, n_remove, rng)

        solved, steps = solve_with_trace(puzzle, max_steps=max_steps)

        # Skip if trace generation failed or produced no steps
        if solved is None or len(steps) == 0:
            continue

        # Verify the trace solution matches the known solution
        if solved != solution:
            continue

        prompt, completion = format_cot_example(puzzle, steps, solution)
        examples.append(json.dumps({"prompt": prompt, "completion":
completion}))

        if len(examples) % 500 == 0:
            print(f"  {len(examples)}/{n} (attempts: {attempts})...")

    print(f"  Generated {n} examples in {attempts} attempts "
          f"({100*n/attempts:.1f}% yield)")
    return examples

def main():
    out_dir = Path(__file__).parent / "data"
    out_dir.mkdir(exist_ok=True)

    print("Generating 5000 CoT train examples (seed=42)...")
    train = generate_cot_examples(5000, seed=42)
    (out_dir / "train_cot.jsonl").write_text("\n".join(train) + "\n")

```

```
print(f"  Written: {out_dir / 'train_cot.jsonl'}")

print("Generating 500 CoT validation examples (seed=99)...")
valid = generate_cot_examples(500, seed=99)
(out_dir / "valid_cot.jsonl").write_text("\n".join(valid) + "\n")
print(f"  Written: {out_dir / 'valid_cot.jsonl'}")

print("Done.")
print()
print("To train:")
print("  Edit finetune/lora_config.yaml: set data: finetune/data")
print("  and update train/valid paths, or copy train_cot.jsonl →
train.jsonl")
print("  python -m mlx_lm.lora --config finetune/lora_config.yaml")
print()
print("To evaluate (server must be running):")
print("  python finetune/eval_sudoku.py")

if __name__ == "__main__":
    main()
```